DESIGN AND IMPLEMENTATION OF A FAIL-SAFE STRING-LEVEL RECONFIGURABLE BATTERY MANAGEMENT SYSTEM FOR A 3S2P LI-ION ROVER PACK

Project Report submitted in the partial fulfilment

of

Bachelor of Technology (B.Tech.)

in

Electronics and Telecommunication Engineering

by

Nihar Lad (D038)
Krish Rajgor (D023)
Harshwardan Patil (D048)

Under the supervision of

Dr. Anjana Rodrigues
(Assistant Professor, Department of Electronics and Telecommunication Engineering, MPSTME)

SVKM's NMIMS University
(Deemed-to-be University)

SVKM'S NMIMS Deemed to be UNIVERSITY

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56
(Academic Year 2026-27)

CERTIFICATE

SVKM'S NMIMS Deemed to be UNIVERSITY

This is to certify that the project entitled "Design and Implementation of a Fail-Safe String-Level Reconfigurable Battery Management System for a 3S2P Li-Ion Rover Pack" has been done by Mr. Nihar Lad (D038), Mr. Krish Rajgor (D023) and Mr. Harshwardan Patil (D048) under the guidance and supervision of Dr. Anjana Rodrigues, and has been submitted in partial fulfilment of the degree of Bachelor of Technology (B.Tech.) in Electronics and Telecommunication Engineering of MPSTME, SVKM's NMIMS (Deemed-to-be University), Mumbai, India.

Dr. Anjana Rodrigues	Examiner
Project Mentor (Name and Signature)	(Name and Signature)

Date:

(HoD) (Name and Signature)
Dr. Avinash More

Place: Mumbai

ACKNOWLEDGEMENT

The project team expresses its sincere gratitude to the project mentor, Dr. Anjana Rodrigues, Department of Electronics and Telecommunication Engineering, MPSTME. Her guidance on the power-switching design, the safety logic and the way results should be verified and reported shaped the direction of this work, and her regular reviews ensured that every claim in this report was checked against its source.

Thanks are due to Dr. Avinash More, Head of the Department of Electronics and Telecommunication Engineering, and to the department for the opportunity and the facilities to carry out this project. The laboratories of the Department of Electronics and Telecommunication Engineering, MPSTME, were used for assembling, wiring and testing the hardware, and the laboratory staff are thanked for their help with instruments, components and access.

The team is also grateful to the families of the team members for their constant support throughout the project.

Student Names

Name	Roll No.	SAP ID
Nihar Lad	D038	
Krish Rajgor	D023	
Harshwardan Patil	D048	

ABSTRACT

Lithium-ion battery packs reach the capacity they need by connecting series strings of cells in parallel. When those strings are hard-wired together, a fault in one string cannot be removed without disconnecting the whole pack, and for a mobile platform that means the mission stops. A switch placed in each string solves this only if the switch blocks current in both directions, never connects two strings at very different voltages, and returns to a safe state whenever its controller fails.

This report presents the design, firmware and hardware validation of a string-level reconfigurable Battery Management System (BMS) for a 3S2P pack of Samsung INR18650-30Q cells that powers a tracked mobile rover. Each of the two strings, and the load, is switched by a bidirectional high-side channel made of two IRF4905 P-channel Metal-Oxide-Semiconductor Field-Effect Transistors (MOSFETs) connected back to back and driven by a discrete NPN level shifter. Base pull-down resistors keep every channel open while the microcontroller is in reset or unpowered. Two INA260 monitors on separate Inter-Integrated Circuit (I^2C) buses measure the voltage and current of each string.

An STM32F446RE microcontroller runs a deterministic 20 ms control loop. A latching safety layer enforces over-voltage, under-voltage, over-current, sensor-loss and stuck-switch limits and may only open switches. A separate reconfiguration state machine is the only code allowed to close a switch, and it does so only after a voltage gate that bounds the true difference between the strings below 100 mV under load, break-before-make confirmed by current readback, and a 500 ms cooldown. Any state it cannot explain forces an all-off lockout that only a reset clears.

The unmodified safety and reconfiguration code passed 22 and 18 host-simulated scenarios and 800 randomized runs without violating any safety invariant. On hardware, the level shifter produced a gate-source voltage of −10.8 V, both sensors reported string voltages with zero bus errors through every switching state, and on the rover an injected over-current isolated the affected string while the other string kept the rover driving in 20 of 20 trials for each string. The absence of per-cell sensing, temperature sensing and a precharge path is stated as a limitation.

Keywords: *Battery management system, Fault isolation, Reconfigurable battery, Back-to-back MOSFET switch, STM32*

Table of Contents

Topics **Page**

Certificate ..... **i**

Acknowledgement ..... **ii**

Abstract ..... **iii**

List of Figures ..... **vi**

List of Tables ..... **vii**

Abbreviations ..... **viii**

Chapter 1 Introduction **1**

1.1 Background 1

1.2 Motivation 2

1.3 Problem Statement 2

1.4 Salient Contributions 2

1.5 Organization of Report 3

Chapter 2 Literature Survey **4**

2.1 Introduction 4

2.2 Literature Survey of Related Work 4

2.3 Comparison of Reviewed Literature 6

2.4 Research Gap 7

2.5 Definition of the Problem 8

Chapter 3 Methodology and Implementation **9**

3.1 Block Diagram 9

3.2 Hardware Description 10

3.3 Software Description 17

3.4 Project Timeline 26

Chapter 4 Results and Analysis **27**

4.1 Testing Approach and Standards Followed 27

4.2 Host Simulation 27

4.3 Switch Stage and Sensing on Hardware 30

4.4 Safety Layer on Hardware 31

4.5 Fault Isolation on the Rover 32

4.6 Engineering Corrections 33

4.7 Discussion of Results 34

Chapter 5 Advantages, Limitations and Applications **36**

5.1 Advantages 36

5.2 Limitations 36

5.3 Applications 37

5.4 Cost of the System 37
5.5 Mapping to Sustainable Development Goals 38
Conclusion and Future Scope 39
Future Scope 39
References 41
Appendix A: Sample Code and Test Log 43
Appendix B: Data Sheets 47
Appendix C: List of Components 49
Appendix D: List of Papers Presented and Published 50

List of Figures

Fig. No.	Name of the Figure	Page No.
1.1	Tracked rover test platform: (a) three-quarter view, (b) side view with the manual kill switch	1
2.1	Classification of approaches to managing multi-string battery packs	4
3.1	Block diagram of the string-level reconfigurable BMS	9
3.2	String A switch channel drawn from the PCB netlist	11
3.3	Effect of the source connection on blocking: (a) incorrect, (b) correct	12
3.4	Prototype stages: (a) sensing and display, (b) back-to-back MOSFET pair and level shifter	16
3.5	Fabricated two-layer switching PCB	16
3.6	Start-up sequence and 20 ms control loop	18
3.7	Logic of the safety layer in each control cycle	20
3.8	Reconfiguration state machine (X: connected string, Y: other string)	22
3.9	Admissible region of the paralleling gate for a maximum path resistance of 50 mΩ (calculated)	23
4.1	Simulated switching sequence after an injected fault on string B	29
4.2	Bench test: (a) both strings on the PCB, (b) OLED status at uptime 00:01:11	31
A.1	Energy, measurement and command flow of the complete system	44

List of Tables

Table No.	Name of the Table	Page No.
2.1	Comparison of reviewed literature	7
3.1	Responsibilities of the subsystems	10
3.2	Design decisions and their reasons	10
3.3	Cell ratings and derived string values	11
3.4	STM32F446RE pin allocation (verified against the PCB netlist)	15
3.5	Firmware modules	17
3.6	Safety-layer rules per string	19
3.7	Guarantees of the reconfiguration module	22
3.8	Operator inputs	24
3.9	Failure modes and effects analysis of the switching hardware and control code	25
3.10	Project timeline (to be completed by the project team)	26
4.1	Mapping of standards and specifications to the project	27
4.2	Safety-layer host-simulation scenarios (simulated)	28
4.3	Reconfiguration host-simulation scenarios (simulated)	28
4.4	Mutation tests on the reconfiguration module (simulated)	30
4.5	Bench measurements compared with calculation	30
4.6	Safety-layer checks on the board (measured / observed)	31
4.7	Fault isolation on the rover (observed)	32
4.8	Bench tests designed but not yet performed	33
4.9	Design errors found and corrected	34
4.10	Status of each subsystem and the evidence behind it	35
5.1	Cost of the components (prices from the project bill of materials)	37
5.2	Mapping of the project to the Sustainable Development Goals	38
B.1	Infineon IRF4905 specifications [12]	47
B.2	onsemi P2N2222A specifications [13]	47
B.3	Texas Instruments INA260 specifications [14]	47
B.4	STM32F446RE and Nucleo-64 board specifications [15, 16]	48
C.1	List of components (from the PCB netlist)	49

Abbreviations

Abbr.	Full Form	Abbr.	Full Form
BMS	Battery Management System	MCU	Microcontroller Unit
BSRR	Bit Set/Reset Register (STM32 GPIO)	MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
CAD	Computer-Aided Design	NPN	Negative-Positive-Negative (bipolar transistor)
CSV	Comma-Separated Values	OCC	Over-Current, Charge
DC	Direct Current	OCD	Over-Current, Discharge
DMM	Digital Multimeter	OLED	Organic Light-Emitting Diode (display)
EV	Electric Vehicle	OV	Over-Voltage
FMEA	Failure Modes and Effects Analysis	PCB	Printed Circuit Board
GPIO	General-Purpose Input/Output	RBS	Reconfigurable Battery System
HAL	Hardware Abstraction Layer	SCL	Serial Clock Line (I^2C)
I^2C	Inter-Integrated Circuit (bus)	SDA	Serial Data Line (I^2C)
IC	Integrated Circuit	SDG	Sustainable Development Goal
IEC	International Electrotechnical Commission	SOA	Safe Operating Area
IEEE	Institute of Electrical and Electronics Engineers	SOC	State of Charge
ISO	International Organization for Standardization	SWD	Serial Wire Debug
IWDG	Independent Watchdog	UART	Universal Asynchronous Receiver-Transmitter
JTAG	Joint Test Action Group (debug interface)	USB	Universal Serial Bus
LED	Light-Emitting Diode	UV	Under-Voltage
Li-ion	Lithium-ion	VCP	Virtual COM Port
LSI	Low-Speed Internal oscillator	ΔV	Voltage difference between the two strings

Chapter 1
Introduction

A battery pack for a mobile robot, an electric vehicle or a storage cabinet is assembled from many small cells. Cells are connected in series to reach the required voltage, and the resulting strings are connected in parallel to reach the required capacity and current. This report starts from a simple question about such packs: what should happen when one of the parallel strings goes wrong, and can a low-cost controller remove that string safely while the rest of the pack keeps working? The answer developed here is a string-level reconfigurable Battery Management System (BMS), built and tested on a tracked rover powered by a 3S2P lithium-ion pack.

This chapter presents the background of the topic, the motivation for the work, the problem statement and the main contributions, and ends with an outline of the remaining chapters. Fig. 1.1 shows the rover test platform.

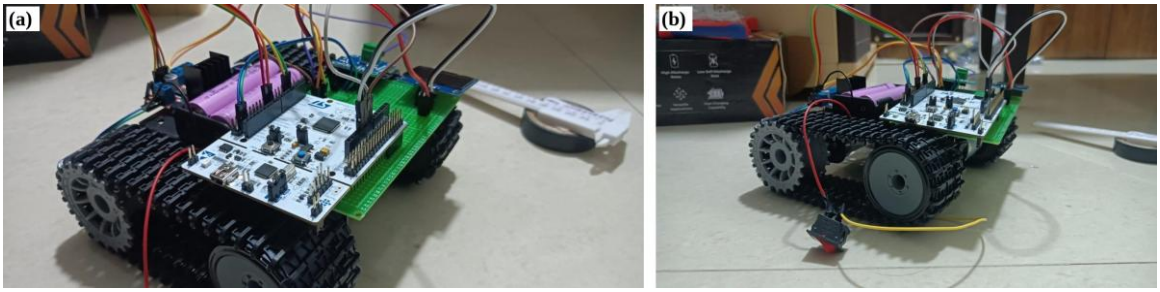


Fig. 1.1: Tracked rover test platform: (a) three-quarter view, (b) side view with the manual kill switch

1.1 Background

A single lithium-ion cell of the 18650 type used in this project has a nominal voltage of 3.6 V and must be kept between 2.5 V and 4.20 V [1]. A pack therefore uses several cells in series to reach a useful voltage, and several such series strings in parallel to provide more capacity and current. In a conventional pack these connections are permanent: the strings are welded or soldered together, and one contactor or switch at the pack terminals connects or disconnects the whole pack.

Permanent connections are simple and have no switching losses, but they tie the behaviour of the pack to its weakest part [2]. When one string develops a fault (an internal short, an abnormal voltage, or an over-current), a fixed pack has only two responses. It can leave the faulted string connected, in which case the healthy strings keep feeding current into or drawing current from it, or it can open the main switch and lose all power. For a vehicle or a robot the second response ends the mission.

Reconfigurable battery systems (RBS) replace some of the fixed connections with switches controlled by the BMS, so that the electrical topology of the pack can change while it is in use [2, 3]. Published work reports benefits such as fault tolerance, charge and temperature balancing, and longer pack life [2, 4], at the cost of extra switching hardware, its losses and its control complexity [3, 5].

1.2 Motivation

The motivation for this project comes from four practical observations.

- **Mission continuity.** A rover that loses one string should be able to keep driving on the other string, at reduced capacity, instead of stopping.
- **The switch itself can create a hazard.** Every power MOSFET contains a body diode, so a single MOSFET cannot block current in both directions. A string that looks disconnected can still exchange current with the rest of the pack. Closing a string onto a bus held at a different voltage is also dangerous, because the only thing limiting the resulting current is a resistance of a few tens of milliohms.
- **Cost and accessibility.** Commercial solutions use dedicated gate-driver and battery-monitor integrated circuits. Showing that a safe string-level switch can be built from discrete, widely available parts makes the approach reproducible in a student laboratory.
- **Verifiable behaviour.** Safety logic for a battery should be simple enough that every switching decision can be traced to a rule and tested. This project therefore uses deterministic, rule-based control rather than learning-based methods.

1.3 Problem Statement

The problem addressed in this project is to design, build and verify a battery management system for a two-string (3S2P) lithium-ion pack on a mobile rover that (i) can disconnect either string from a live bus in both current directions, (ii) never connects the two strings in parallel unless their sensed voltage difference is below 100 mV, (iii) proves that a switch has actually opened before another one closes, (iv) defaults to all switches open whenever the controller is reset, unpowered or hung, and (v) keeps the rover running on the healthy string when one string is isolated.

1.4 Salient Contributions

- A bidirectional high-side string switch built from two back-to-back IRF4905 P-channel MOSFETs and a 2N2222A-family NPN level shifter, with design calculations from datasheet values and a hardware default-off state.
- Identification and correction of a topology error (MOSFET sources tied to the battery rail) that passes an ordinary on/off test but leaves one current direction unblocked.
- A layered firmware architecture on an STM32F446RE in which the safety layer can only open switches and a separate reconfiguration module is the only code allowed to close one, under five enforced guarantees.
- A paralleling gate that bounds the true voltage difference between the strings under load without knowing the exact path resistance.
- Verification at three separate evidence levels: calculation, host simulation of the unmodified firmware with fuzzing and mutation testing, and bench and rover tests on a fabricated printed circuit board (PCB).

1.5 Organization of Report

Chapter 2 reviews published work on reconfigurable batteries and fault-tolerant battery management, compares it, and identifies the gap this project addresses. Chapter 3 describes the hardware and firmware: the system block diagram, the switch channel and its calculations, the sensing, the PCB and rover, the control loop, the safety layer, the reconfiguration state machine and the failure analysis. Chapter 4 presents the testing approach and the simulation, bench and rover results. Chapter 5 discusses the advantages, limitations and applications of the system, its cost and its relation to the Sustainable Development Goals. The report closes with the conclusions, future scope, references and appendices containing sample code, datasheet extracts, the component list and the paper prepared from this work.

Throughout the report each result is labelled by its source: *calculated* values come from equations with stated inputs, *simulated* values come from running the firmware on a computer against a model, and *measured* or *observed* values come from the fabricated hardware. No simulated value is presented as a hardware result.

Chapter 2
Literature Survey

This chapter reviews published work relevant to the project, compares the reviewed systems, identifies the gap that remains, and defines the problem in terms of that gap.

2.1 Introduction

Work on multi-string battery management falls into three broad groups, shown in Fig. 2.1. The first group keeps the pack topology fixed and manages it through monitoring, protection and balancing. The second group makes the topology reconfigurable, with switches at the level of cells, modules or whole strings, and studies the hardware and the control algorithms needed. The third group addresses the estimation problems that reconfiguration creates, such as tracking the state of charge (SOC) of a pack whose structure changes. The literature reviewed below was chosen to cover all three groups and to provide the component and safety references on which the design relies.

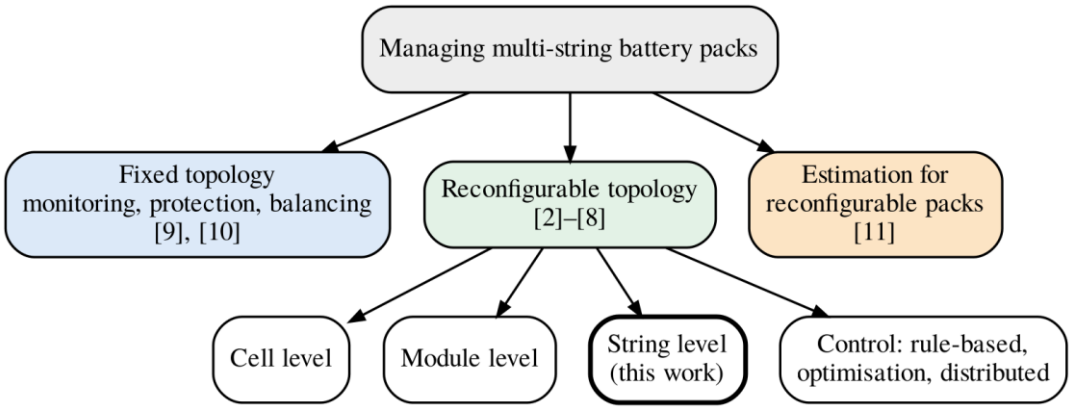


Fig. 2.1: Classification of approaches to managing multi-string battery packs

2.2 Literature Survey of Related Work

2.2.1 Surveys of reconfigurable battery systems

Han *et al.* [2] survey next-generation battery management systems with dynamic reconfiguration. They list the benefits of reconfigurable systems as enhanced fault tolerance, charge and temperature balancing, extended energy delivery, the ability to combine batteries of different age or chemistry, and customised output voltage ranges, and they review the hardware designs, management principles and optimisation algorithms used to achieve them. Their discussion of fault tolerance is directly relevant here: in a fixed configuration, a cell fault can spread to neighbouring cells and escalate to pack level.

Steinhorst *et al.* [3] review battery system management architectures and identify two trends: decentralised management, in which computation moves closer to the cells, and reconfigurable topologies, in which the series and parallel connections can change at run time. They note that reconfiguration could increase reliability and reduce balancing effort, but that the limited energy efficiency of the additional hardware is a challenge.

Tang *et al.* [5] survey reconfigurable battery topologies and classify them by the number of switching devices required per cell, compare their performance metrics and identify suitable applications. This classification makes clear that fine-grained, cell-level reconfiguration multiplies the number of switches, which motivates the coarser string-level approach chosen in this project.

Pinter *et al.* [6] review the control algorithms used in reconfigurable battery systems. They group them by control strategy and by implementation aspects such as reconfiguration frequency, balancing operation and control topology, and describe an industrial reconfigurable BMS as an example.

2.2.2 System-level benefit of reconfiguration

Škegro *et al.* [4] assess dynamic reconfiguration for electric-vehicle packs with a statistical framework based on detailed cell modelling. They report that reconfigurable packs can extend battery lifetime by more than 20%, particularly in high-voltage applications, and identify the conditions under which reconfiguration is economically viable. These figures are their modelled results for large vehicle packs; they are quoted here only as motivation and are not claimed for the small pack built in this project.

2.2.3 Reconfiguration control and fault isolation

Morstyn *et al.* [7] present distributed control for SOC balancing between the modules of a reconfigurable battery energy storage system. Autonomous modules share SOC estimates with their neighbours and cooperate to balance, and a current controller prevents current spikes during module topology transitions. Their experiments use four 6 V, 7 Ah lead-acid modules. The concern for current spikes at topology changes is the same concern that leads to the paralleling gate in this project.

Lemeon and Regash [8] propose a reconfigurable battery pack architecture for fault isolation and lifetime extension, using a modular switching topology with fault detection and isolation logic that monitors cell voltage and SOC. The design is simulated in

MATLAB/Simulink under normal, faulted and post-reconfiguration conditions, and the authors report sub-second fault detection and improved SOC balancing in simulation.

2.2.4 Fault-tolerant battery management hardware

Canilang *et al.* [9] design a modular, fault-tolerant BMS with a central management unit and local management units, and three levels of isolation: module-level, component-level and ground-level. They present both simulation and a hardware prototype tested under faulty scenarios.

Gao *et al.* [10] implement real-time fault diagnosis and protection for a lithium-ion battery system on a 12-cell series pack prototype, including SOC self-initialisation, fault diagnosis and self-recovery actions under an electric-vehicle load profile. Their pack is fixed; protection acts on the pack as a whole.

2.2.5 State estimation for reconfigurable modules

Zhao *et al.* [11] address SOC estimation in reconfigurable battery modules, where frequent topology changes cause conventional estimators to accumulate error. They propose a hybrid Gated Recurrent Unit and Transformer model that recognises topology changes. This line of work shows that estimation in reconfigurable packs is an active problem; the present project deliberately keeps its switching decisions independent of any learned model.

2.2.6 Component and safety references

The design values in this project are taken from manufacturer documents: the cell specification [1], the MOSFET [12] and NPN transistor [13] datasheets, the current monitor datasheet [14], the microcontroller datasheet [15] and the development-board manual [16]. The structure of the safety analysis follows the vocabulary of the functional-safety standards ISO 26262 [17] and IEC 61508 [18], without any claim of compliance.

2.3 Comparison of Reviewed Literature

Table 2.1 compares the reviewed work with this project. Only properties stated in each publication are listed.

Table 2.1: Comparison of reviewed literature

Ref.	Focus	Level / scope	Validation
[2]	Dynamically reconfigurable BMS	Cell, module and pack	Survey
[3]	Battery management architectures	Distributed and reconfigurable	Survey
[5]	Reconfigurable topologies	Classified by switches per cell	Survey
[6]	Reconfiguration control algorithms	Strategy and implementation	Survey with an industrial example
[4]	Lifetime and cost of reconfiguration	Electric-vehicle packs	Statistical modelling
[7]	Distributed SOC balancing	Module	Experiment, four 6 V lead-acid modules
[8]	Fault isolation and lifetime	Modular switching	MATLAB/Simulink simulation
[9]	Fault-tolerant modular BMS	Module, component and ground isolation	Simulation and prototype
[10]	Real-time fault diagnosis	Fixed 12-cell series pack	Prototype
[11]	SOC estimation	Reconfigurable modules	Experimental data
This work	Fault isolation with discrete switches	String (two 3S strings)	Firmware simulation, bench, rover trials

2.4 Research Gap

Taken together, the reviewed work leaves the following gaps.

- **Switch-level hazards are rarely treated in detail.** Surveys describe reconfigurable topologies, but the body-diode conduction of a single MOSFET and the inrush that follows closing onto a mismatched bus are seldom analysed at the level of a concrete circuit.
- **Many evaluations are simulation-only or use large converters.** Reconfiguration is often demonstrated in simulation or with power converters and dedicated ICs, which hides the behaviour of simple discrete switches.
- **Separation of protection and reconfiguration authority is not explicit.** Where control logic both protects and reconfigures, an error in the reconfiguration logic can undo a protective action.
- **Evidence levels are mixed.** Results from models, calculations and hardware are often reported together, which makes it hard to know what has actually been demonstrated on hardware.

This project addresses these gaps with a string-level switch built from discrete parts, an explicit analysis of its hazards, firmware in which only one module may close a switch and

only under checked conditions, and results reported separately by evidence level, ending with fault isolation on a moving vehicle.

2.5 Definition of the Problem

From the gaps above, the problem addressed by this project is defined as follows: to build a string-level reconfigurable BMS for a 3S2P lithium-ion rover pack, using discrete back-to-back P-channel MOSFET switches and a general-purpose microcontroller, that isolates a faulted string in both current directions while the other string continues to supply the load, never parallels strings across an unsafe voltage difference, defaults to all-off on any controller failure, and is verified by calculation, by simulation of the unmodified firmware and by tests on fabricated hardware.

This chapter reviewed reconfigurable battery systems, fault-tolerant battery management and estimation for reconfigurable packs, compared the most relevant work and identified the gaps. Chapter 3 now describes how the system was designed and built to address them.

Chapter 3
Methodology and Implementation

This chapter describes the complete system: the block diagram, the hardware from the cells to the rover, and the firmware that runs on the microcontroller. Design values are shown with their source, and every pin and connection was checked against the netlist exported from the KiCad schematic of the fabricated board.

3.1 Block Diagram

Fig. 3.1 shows the block diagram. Two strings, A and B, each of three cells in series, reach a common node called BUS through their own switch pair. A third pair connects BUS to the motor driver. Because the strings join only after their switches, opening one pair removes that string from the bus while the other continues to feed the load. Each string has its own current and voltage monitor, and the microcontroller drives each switch pair through a level shifter.

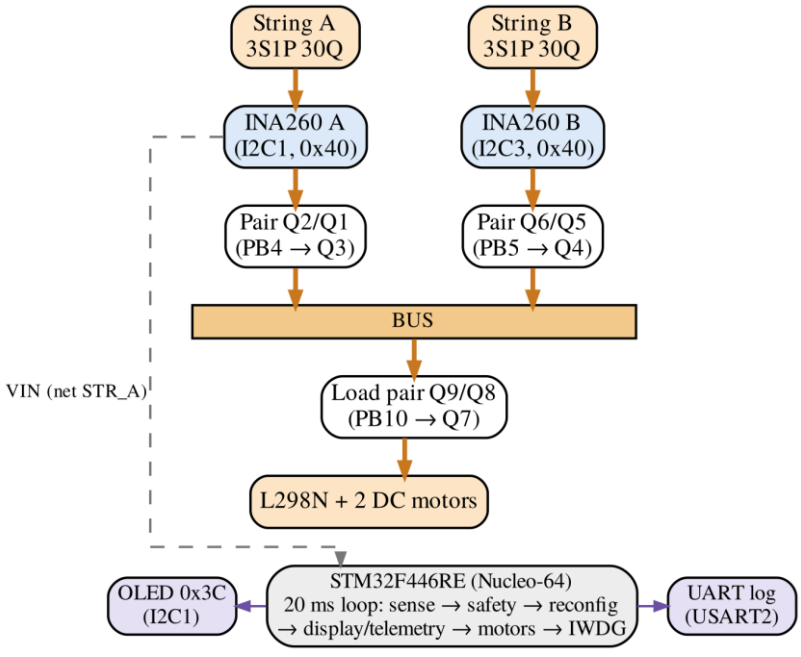


Fig. 3.1: Block diagram of the string-level reconfigurable BMS

Table 3.1 lists the responsibility of each subsystem. The division matters for safety: only the reconfiguration module may turn a switch on, while the safety layer, the error handler and the hardware pull-downs can only turn switches off.

Table 3.1: Responsibilities of the subsystems

Subsystem	Responsibility	Can close a switch?
Back-to-back MOSFET pairs and level shifters	Connect or isolate each string and the load	—
Base pull-down resistors	Hold every pair open when the MCU does not drive the pins	No
INA260 monitors	Measure voltage and current of each string	No
Safety layer	Enforce the safe operating area and latch faults	No
Reconfiguration state machine	Choose the topology and sequence the switching	Yes (the only one)
Error handler and watchdog	Force all switches off on a software failure	No
Display and telemetry	Report state, faults and counters	No
Motor gate	Run the motors only while a string feeds the load	No

3.1.1 Design decisions

Several choices were fixed early and shaped the rest of the design. They are listed in Table 3.2 with the reason for each, so that the later sections can be read against them.

Table 3.2: Design decisions and their reasons

Decision	Reason
Switch whole strings, not cells	Two switch pairs per pack instead of one or more per cell; fewer devices and lower conduction loss
No per-cell bypass	Bypassing one cell would turn a 3S string into a 2S string, which cannot be connected in parallel with a 3S string
No DC-DC converter between strings and bus	The strings are connected directly through switches; a converter would add loss, cost and a second control problem
High-side P-channel MOSFETs with an NPN level shifter	The gate can be driven from the pack voltage itself; no gate-driver IC, bootstrap or charge pump is needed
Back-to-back MOSFET pairs	A single MOSFET conducts through its body diode in one direction even when off
Both strings in parallel in normal operation	Reconfiguration is used for faults; in normal operation the load is shared
Rule-based control, no machine learning	Every switching decision can be traced to a rule and tested deterministically
Two I²C buses for the sensors	Both INA260 modules are fixed at the same address

3.2 Hardware Description

3.2.1 Battery pack

The pack consists of six Samsung INR18650-30Q cells arranged as two 3S1P strings. Table 3.3 lists the cell ratings from the manufacturer specification [1] and the string values

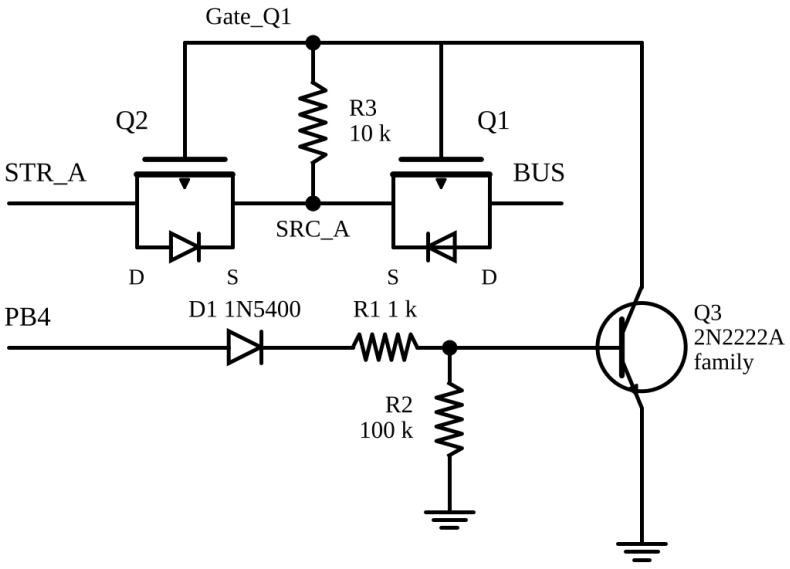
derived from them. The string limits (12.60 V maximum, 7.50 V minimum) become the over-voltage and under-voltage thresholds of the safety layer in Section 3.3.3.

Table 3.3: Cell ratings and derived string values

Quantity	Value	Source
Cell nominal voltage	3.6 V	[1] §3.2
Cell charge voltage	4.20 ± 0.05 V	[1] §3.3
Cell discharge cut-off	2.5 V	[1] §3.8
Maximum continuous discharge current	15 A (25 °C)	[1] §3.7
Rated charge current	4 A	[1] §3.4
Standard discharge capacity	≥ 2950 mAh	[1] §7.3
Initial internal impedance (AC 1 kHz)	≤ 26 mΩ	[1] §7.5
String nominal / maximum / minimum voltage	10.8 / 12.6 / 7.5 V	Calculated (×3)

3.2.2 Bidirectional high-side switch channel

Each of the three switch channels uses two IRF4905 P-channel MOSFETs [12]. Fig. 3.2 shows the string A channel as drawn from the netlist. The drain of Q2 connects to the string terminal (net STR_A), the drain of Q1 connects to BUS, and the two sources are joined at a node (net SRC_A) that connects to nothing except the two sources and the 10 kΩ gate pull-up R3. The two gates are tied together (net Gate_Q1). String B (Q6, Q5, SRC_B) and the load channel (Q9, Q8, SRC_load) are identical.



Default OFF: R2 holds the base low when PB4 floats.

Fig. 3.2: String A switch channel drawn from the PCB netlist

Each MOSFET contains a body diode from drain to source. In this arrangement the two body diodes point towards the shared source node. With both MOSFETs off, current from the string forward-biases the diode of Q2 but meets the diode of Q1 in reverse; current from the bus forward-biases the diode of Q1 but meets the diode of Q2 in reverse. The pair therefore blocks in both directions, which a single MOSFET cannot do. When the gates are pulled low, both channels conduct and the body diodes are bypassed.

This arrangement was reached only after an error that recurred in the breadboard prototype, in the first schematic and again after a partial fix. In the incorrect version, shown in Fig. 3.3(a), both sources were tied to the battery rail. One MOSFET was then bypassed by copper, and the remaining one had its body diode pointing from the bus into the string. The circuit still switched the load on and off, so a functional test passed, but an isolated string could still receive current from the bus. The error was finally caught by reading the netlist rather than the schematic: a correct channel produces a three-node source net containing exactly two MOSFET sources and one resistor.

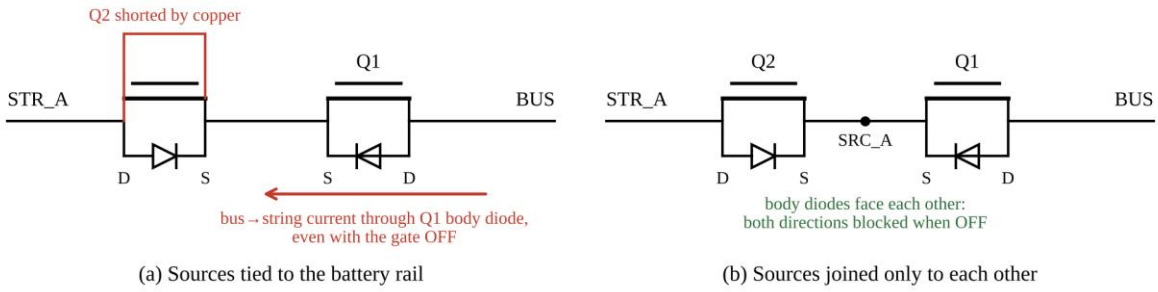


Fig. 3.3: Effect of the source connection on blocking: (a) incorrect, (b) correct

3.2.3 Level-shifter gate drive

The microcontroller output is 0 V or 3.3 V, but the IRF4905 needs its gate about 10 V below its source to conduct fully. An NPN transistor of the 2N2222A family performs the level shift. When the control pin (PB4 for string A) is high, base current flows through the 1N5400 diode D1 and the 1 kΩ resistor R1, the transistor saturates, and its collector pulls the shared gate net down. Using typical junction drops of 0.6 V for the diode and 0.7 V for the base-emitter junction, and a string voltage of 11.9 V (all values calculated):

$$I_B = (3.3 - 0.6 - 0.7) / 1\ \text{k}\Omega - 0.7 / 100\ \text{k}\Omega \approx 1.99\ \text{mA} \quad (3.1)$$

$$I_{C,sat} = (11.9 - 0.3) / 10\ \text{k}\Omega = 1.16\ \text{mA} \quad (3.2)$$

The second term of (3.1) is the current diverted by the 100 kΩ base pull-down R2. In (3.2), 0.3 V is the collector-emitter saturation voltage, which the datasheet specifies as a

maximum at a much higher collector current of 150 mA [13]; it is therefore used here as an upper bound. The datasheet guarantees a minimum DC gain of 50 at a collector current of 1 mA and a collector-emitter voltage of 10 V [13], so the transistor can sink up to about 50 × 1.99 mA ≈ 99.5 mA while the circuit needs only 1.16 mA. This factor of about 86 drives the transistor deep into saturation, and the gate-source voltage of the MOSFETs becomes

$$V_{GS(on)} = V_{CE(sat)} - V_{SRC} \approx 0.3 - 11.9 = -11.6\ \text{V} \tag{3.3}$$

where V_{SRC} is the voltage of the shared source node, equal to the string voltage when the channel is on. This value lies beyond the worst-case threshold of −4.0 V and beyond the −10 V at which the on-resistance is specified [12]. At a full string of 12.6 V it becomes about −12.3 V, inside the ±20 V gate rating. At the 7.5 V under-voltage limit it falls to about −7.2 V; the MOSFET still conducts, but its on-resistance is then higher than the datasheet maximum, which is allowed for in the paralleling gate of Section 3.3.5.

When the control pin is low, no current flows through R3, so the gate sits at the same potential as the source and V_{GS} = 0 V. R3 returns to the floating source node rather than to the battery rail; returning it to the rail would leave V_{GS} at about +0.7 V, because the source node settles one body-diode drop below the string. The same resistor determines how quickly the pair turns off, since it alone removes the gate charge. With the datasheet maximum gate charge Q_g = 180 nC per device [12], specified at a higher drain current and voltage and therefore an upper bound,

$$t_{off} \approx 2Q_gR_3 / |V_{GS}| = 2 \times 180\ \text{nC} \times 10\ \text{k}\Omega / 10\ \text{V} \approx 0.36\ \text{ms} \tag{3.4}$$

which is about 55 times shorter than one 20 ms control cycle. The firmware does not rely on this estimate for break-before-make; it proves that a string is open by reading its current (Section 3.3.6).

The steady power in R3 when the channel is on is $(11.6\ \text{V})^2 / 10\ \text{k}\Omega \approx 13.5$ mW, well within the rating of a 1/4 W resistor. Conduction loss in a closed pair is $I^2 \times 2R_{DS(on)}$; with the 20 mΩ maximum per device and a 1.19 A load (a 10 Ω test resistor at 11.9 V), it is $1.19^2 \times 0.040 \approx 57$ mW per pair (calculated).

The level shifter also explains why the first prototype, powered entirely from the 3.3 V rail, did not switch at all: with a 3.3 V source, $V_{GS} = 0.3 - 3.3 = -3.0$ V, which lies inside the −2.0 V to −4.0 V threshold range [12], so conduction was not guaranteed. Powering the switch stage from the pack while keeping the control pins at 3.3 V solved this.

3.2.4 Thermal check of the switches

Each closed MOSFET dissipates $I^2R_{DS(on)}$. With the datasheet junction-to-ambient thermal resistance of 62 °C/W for a device in free air without a heatsink [12], the junction temperature rise is

$$\Delta T_J = I^2 \times R_{DS(on)} \times R_{\theta JA} = I^2 \times 0.02\ \Omega \times 62\ °C/W \quad (3.5)$$

At the 1.19 A test load this is about 1.8 °C, which is negligible. The picture changes at high current. At the 12 A discharge trip the dissipation is $12^2 \times 0.02 \approx 2.9$ W per device and the calculated rise is about 179 °C, which would take the junction past its 175 °C maximum [12] if sustained. Allowing a 150 °C rise from a 25 °C ambient limits the continuous current to about $\sqrt{(150 / (0.02 \times 62))} \approx 11$ A per string, and the real limit is lower because the on-resistance increases with temperature. A current just below the 12 A trip would therefore not be caught by the safety layer but would overheat the switches if it lasted. This is not reached on the rover, whose drivetrain draws far less, but any use near the trip current requires heatsinks or a lower trip threshold (all values calculated).

3.2.5 Fail-safe default state

During reset, start-up or brown-out the microcontroller pins are high-impedance. The 100 kΩ base pull-down of each NPN (R2, R5, R8) then holds the transistor off, so every pair is open and every string is disconnected without any software action. In firmware, the switch pins are written low through the GPIO Bit Set/Reset Register (BSRR) before they are configured as outputs, so the port never drives an undefined level, and the error handler forces all three pins low with a single register write before disabling interrupts, without calling any library function that might itself have failed (Appendix A).

PB4, which controls string A, needs particular attention. At reset it is the JTAG NJTRST pin with an internal pull-up, so until the firmware reconfigures it the microcontroller itself can source base current into the string A channel. A worst-case calculation with the weakest internal pull-up shows that this current could be enough to saturate the NPN. A bench test with the reset button held showed the string A drive circuit below the NPN turn-on level and string A off (Section 4.4). Because the analytical margin is small, a 2.2 kΩ pull-down on this channel, or moving the channel to a pin without a debug function, is recommended for a hardware revision. The same pin, and PB3, also do not work as general-purpose outputs unless the debug interface is set to Serial Wire Debug (SWD) only, a configuration trap that initially looked like a hardware fault.

3.2.6 Current and voltage sensing

Each string has a Texas Instruments INA260 power monitor with an integrated 2 mΩ shunt, a current resolution of 1.25 mA and a bus-voltage resolution of 1.25 mV [14]. The breakout modules used have their address pins fixed, so both respond at address 0x40. Rather than adding an address translator, the two sensors were placed on separate I²C peripherals of the microcontroller: sensor A on I2C1 (PB8/PB9), which it shares with the 128×64 OLED display at address 0x3C, and sensor B on I2C3 (PA8/PC9). Each sensor is wired into its string with external leads on the string side of the switch, so that it reads its own string voltage even while that string is disconnected. This placement is a precondition for the paralleling gate.

The sensors replaced an earlier plan that used resistor dividers into the microcontroller's analogue inputs for each cell and a separate current sensor. The change simplified the hardware but means that only string voltage is measured, not individual cell voltages; Chapter 5 treats this as a limitation.

3.2.7 Controller and pin allocation

The controller is an STM32F446RE microcontroller [15] on a Nucleo-64 development board [16]. Table 3.4 lists the pin allocation, verified against the netlist of the fabricated board. The Nucleo VIN pin is tied to net STR_A, upstream of the string A switch, so the controller remains powered when string A is switched off.

Table 3.4: STM32F446RE pin allocation (verified against the PCB netlist)

Function	MCU pin	Connection
String A switch	PB4	D1 → R1 → base of Q3; Q3 collector on net Gate_Q1 (Q1, Q2)
String B switch	PB5	D2 → R4 → base of Q4; Q4 collector on net Gate_Q5 (Q5, Q6)
Load switch	PB10	D3 → R7 → base of Q7; Q7 collector on net Gate_Q8 (Q8, Q9)
I2C1 SCL / SDA	PB8 / PB9	INA260 A (U2) and OLED (DS1)
I2C3 SCL / SDA	PA8 / PC9	INA260 B (U3)
Motor IN1 / IN2	PA9 / PA10	L298N module (U4) pins 7, 8
Motor IN3 / IN4	PB3 / PC7	L298N module (U4) pins 9, 10
Telemetry UART	PA2 / PA3	USART2 to the on-board ST-Link virtual COM port
Drive button	PC13	On-board user button
Heartbeat LED	PA5	On-board LED
Board supply	VIN (CN6 pin 8)	Net STR_A, upstream of the string A switch

3.2.8 Prototype stages and printed circuit board

The hardware was developed in three stages. A perfboard carrying the Nucleo board, one INA260 and the OLED was used to bring up sensing and the display, and a second perfboard carried a discrete back-to-back MOSFET pair and level shifter to bring up switching (Fig. 3.4). These prototypes exposed the gate-drive and topology errors described above before any board was fabricated.

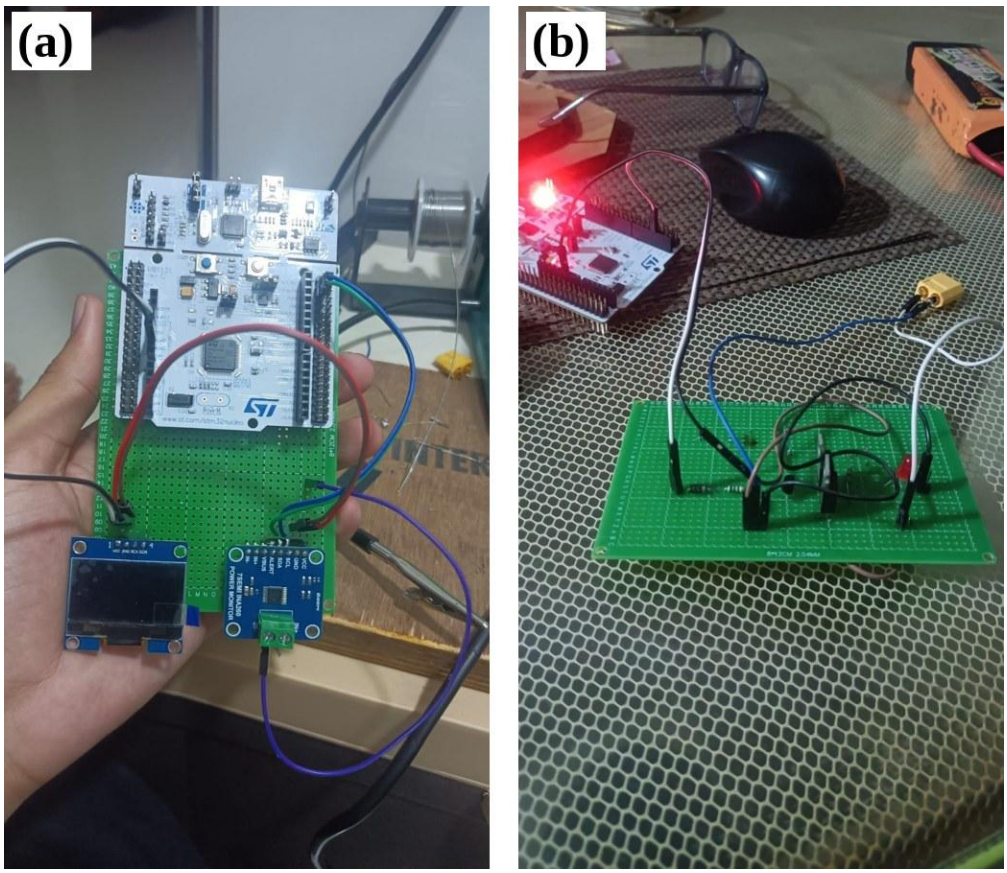


Fig. 3.4: Prototype stages: (a) sensing and display, (b) back-to-back MOSFET pair and level shifter

The final circuit was laid out as a two-layer PCB (Fig. 3.5). It carries the Nucleo-64 board, six IRF4905 MOSFETs in three pairs, three NPN level shifters with their base and gate resistors and base diodes, the two INA260 modules, the OLED and the L298N motor-driver module. Screw terminals J1 and J3 connect strings A and B. The netlist confirms that the precharge control pin (PB6) is not connected: the fabricated board has no precharge path, so paralleling relies on the voltage gate alone.

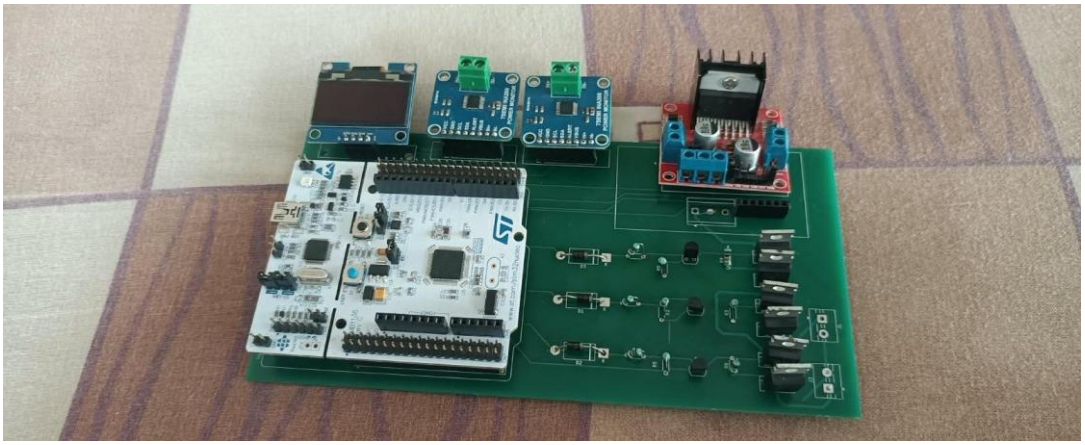


Fig. 3.5: Fabricated two-layer switching PCB

3.2.9 Rover platform

The test platform is a tracked, differential-drive rover with two brushed DC motors driven by an L298N dual H-bridge module (Fig. 1.1). The motor-driver inputs IN1 to IN4 are driven from PA9, PA10, PB3 and PC7, and the module's 12 V input is the switched load net. A manual kill switch in the power path allows the operator to cut power independently of the controller. The drivetrain draws far less than the 12 A over-current limit, so over-current faults on the rover were emulated by fault injection (Section 3.3.3).

3.3 Software Description

3.3.1 Firmware structure

The firmware is written in C for the STM32 Hardware Abstraction Layer (HAL) and is divided into modules with narrow responsibilities, listed in Table 3.5. Dependencies run in one direction only: the reconfiguration module uses the safety and switch modules, but no other module includes the reconfiguration module. The display and telemetry modules receive the topology only as a text label.

Table 3.5: Firmware modules

Module	Responsibility
main.c	Start-up order, 20 ms loop, watchdog start and refresh, reset-cause report
bms_switch.c	GPIO for the three pairs; all-off in a single BSRR write
i2c_bus.c	Timeouts, retries and nine-pulse bus recovery, separately for each bus
ina260.c	Register access, conversion to millivolts and milliamps, identity probe
bms_safety.c	Safe-operating-area rules, debounce, latching, enforcement, block mask, injection hook
reconfig.c	Topology state machine; the only code that closes a switch
oled.c, bms_display.c	OLED driver and eight-row status screen, one page per loop
telemetry.c	CSV lines at 5 Hz and event lines over USART2
motor.c	L298N direction pins, gated by load availability
bench_debug.c	Operator commands: emergency stop and fault injection

3.3.2 Deterministic control loop

The firmware is a tick-scheduled loop with a fixed 20 ms period and a fixed order of operations (Fig. 3.6). Safety runs immediately after sensing and before anything that can close a switch, and the watchdog is refreshed last, so only a cycle that has completed sensing and safety earns more time.

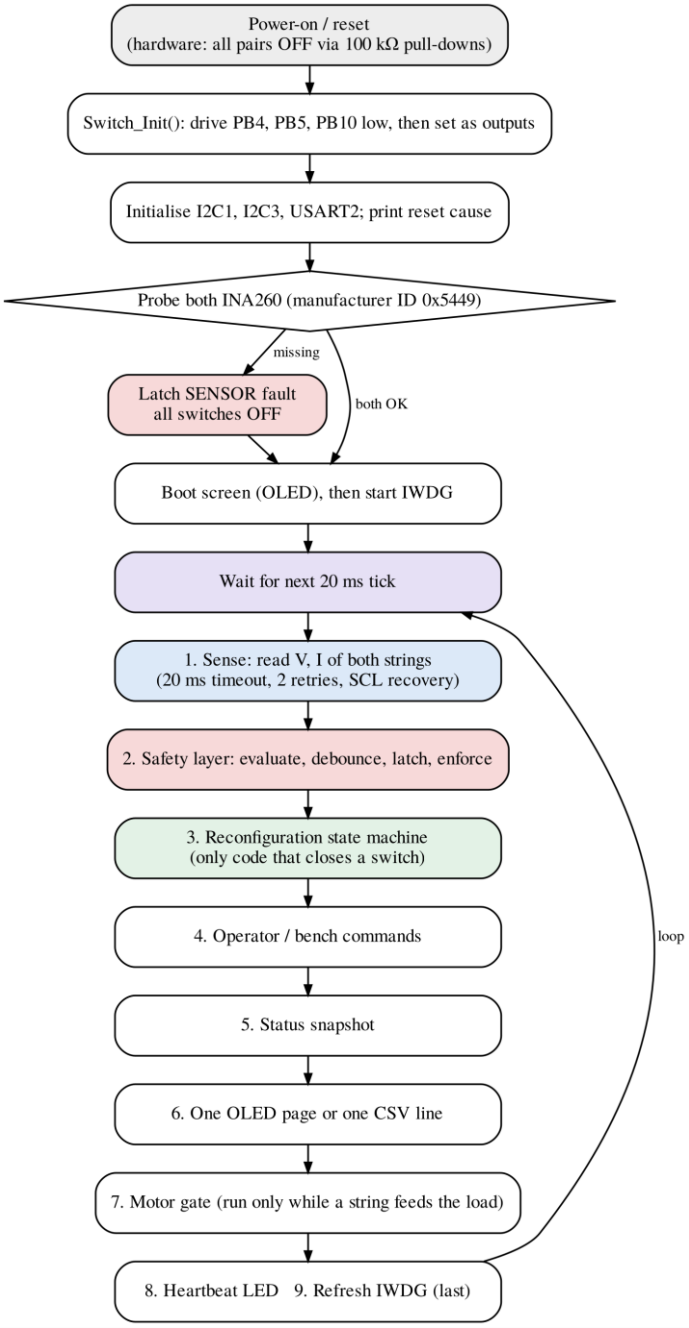


Fig. 3.6: Start-up sequence and 20 ms control loop

Every I²C transaction has a 20 ms timeout and two retries. If the retries fail, the driver de-initialises the peripheral, drives SCL manually for nine clock pulses to release a slave that is holding SDA low, and re-initialises; this is the bus-clear procedure of the I²C specification [19]. Each bus recovers independently. Sensor functions return a status code separately from the measured value. An earlier version signalled failure by returning −9999 in the current field, which corresponds to a legitimate −10 A reading within the sensor range and would have hidden a real reverse-current fault.

The independent watchdog (IWDG) runs from the internal low-speed (LSI) oscillator with prescaler $PR = 32$ and reload $RLR = 499$:

$$t_{WDG} = (RLR + 1) \times PR / f_{LSI} = 500 \times 32 / f_{LSI} \quad (3.6)$$

which is 0.5 s at the nominal 32 kHz and about 0.34 s to 0.94 s over the oscillator tolerance range given in the device datasheet [15]. The watchdog is started by direct register writes after the boot screen, so that the boot screen does not outlast the timeout, and it is frozen while a debugger halts the core. A hung loop ends in a reset, and a reset ends in the hardware all-off state. At every boot the firmware reports the reset cause (power-on, reset pin, watchdog or brown-out).

3.3.3 Safety layer

The safety layer enforces the safe operating area (SOA) of each string. Its rules are listed in Table 3.6 and its logic is shown in Fig. 3.7. Voltage limits are three times the cell limits of Table 3.3; the discharge limit of 12 A is set 3 A below the cell's maximum continuous rating, and the charge limit equals the rated charge current [1]. Because only string voltage is measured, the voltage limits act as average-per-cell limits.

Table 3.6: Safety-layer rules per string

Fault	Condition	Class	Response
OV	$V > 12.60$ V	String	Open the string, latch
UV	$V < 7.50$ V	String	Open the string, latch
OC discharge	$I > 12$ A	String	Open the string, latch
OC charge	$I > 4$ A	String	Open the string, latch
Sensor lost	Read fails after all retries	Global	All off, latch
Stuck switch	> 200 mA in an OFF string, 100 ms after the OFF command	Global	All off including load, latch

Debounce of three consecutive samples applies to all rules except sensor loss, which acts at once because the I²C layer has already spent its retries.

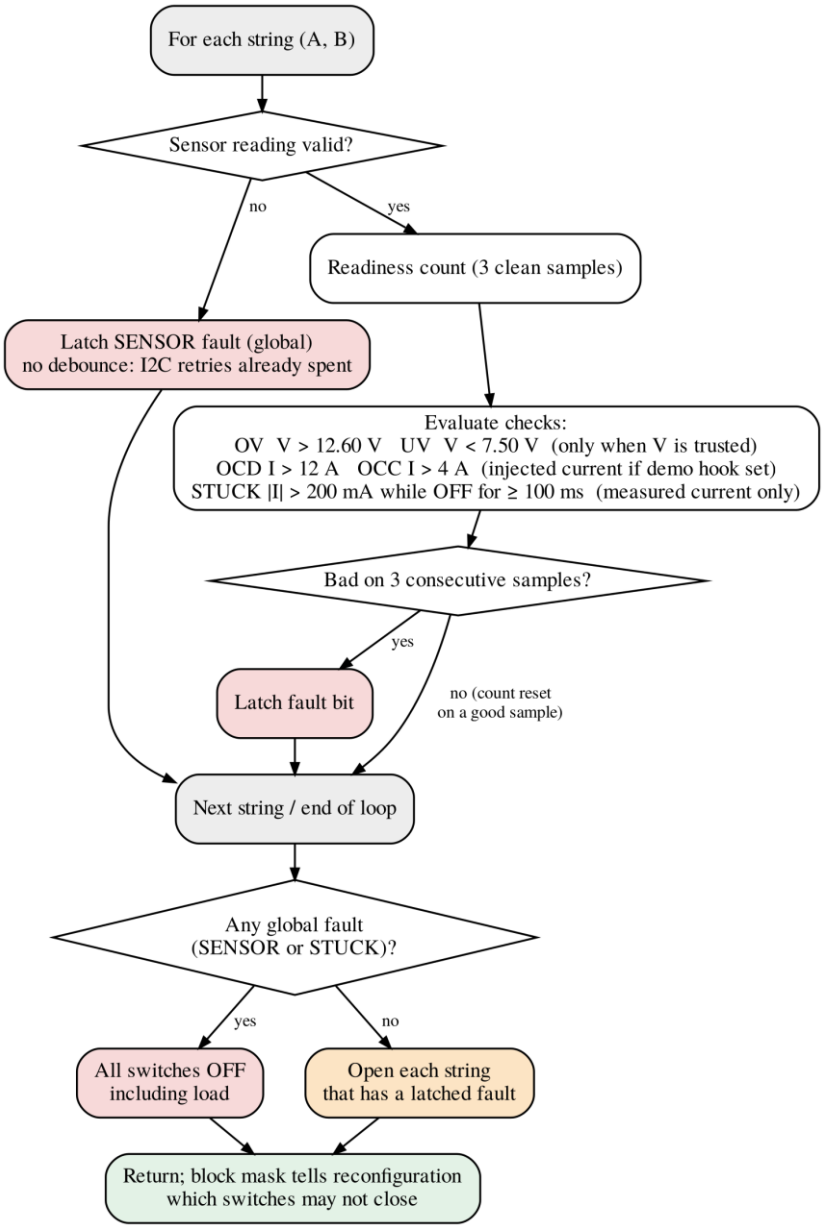


Fig. 3.7: Logic of the safety layer in each control cycle

Five rules govern the layer.

- **Debounce before acting.** A limit must be exceeded on three consecutive samples; one good sample resets the count. This filters motor transients shorter than about 60 ms.
- **Latch, never self-clear.** A latched fault stays latched after the condition disappears, so a fault that clears itself can never reconnect a string onto the same problem.
- **Reset is the only exit.** Faults live in memory and there is no clear-fault command. A reset only makes a string eligible again; it reconnects nothing.
- **An unsensed pack is an unsafe pack.** If a sensor stops answering, the controller can no longer confirm that the unseen string is open, so everything is switched off.

- **A stuck switch disables everything.** If a pair carries current while commanded off, it cannot be opened by firmware, so the load switch, the only remaining break, is opened too.

The calculated trip latency for a sustained limit violation is

$$t_{\text{trip}} = 3 \times 20\ \text{ms} + (0 \ldots 20\ \text{ms}) = 60 \ldots 80\ \text{ms} \tag{3.7}$$

where the second term is the phase of the fault relative to the sampling instant. This is far too slow to interrupt a hard short circuit, so short-circuit protection must be provided by a fuse in each string. The layer re-applies its decisions in every cycle: if any other code turns a faulted string back on, it is forced off again within 20 ms. Any code that wishes to close a switch must first consult the layer's block mask. After any reset, no switch may close until each sensor has delivered three valid samples, so a fault that is still present re-latches before anything connects.

Voltage limits are judged only when the sensor reading is known to be that string's own voltage. In the build used for paralleling, where the sensors were confirmed to read string-side voltage, this is always the case; in the single-string build, a string's voltage is trusted only after it has been on for 100 ms. Without this rule, an open string's sensor could read an unpowered bus at 0 V at start-up, latch under-voltage on both strings, and leave a system that could never start.

For demonstration, a fault-injection hook replaces the current seen by the safety layer for one string with a value 1 A above the discharge limit. Debounce, latching and switching then run on the normal code path, while the display and the log still show the measured current and mark the fault label with an asterisk. The stuck-switch check always uses the measured current, so injection cannot fake a stuck switch, and the hook can be removed from the build with a single setting.

3.3.4 Reconfiguration state machine

The reconfiguration module (Fig. 3.8) runs immediately after the safety layer in each cycle. It is the only code that turns a switch on, and it never turns on a switch that the safety layer blocks; it never needs to turn a switch off for protection, because the safety layer does that. Each step begins by comparing the commanded switch states with the GPIO output register: an opening performed by the safety layer is accepted as a fact, while any other difference means that some other code moved a switch, and the module enters LOCKOUT.

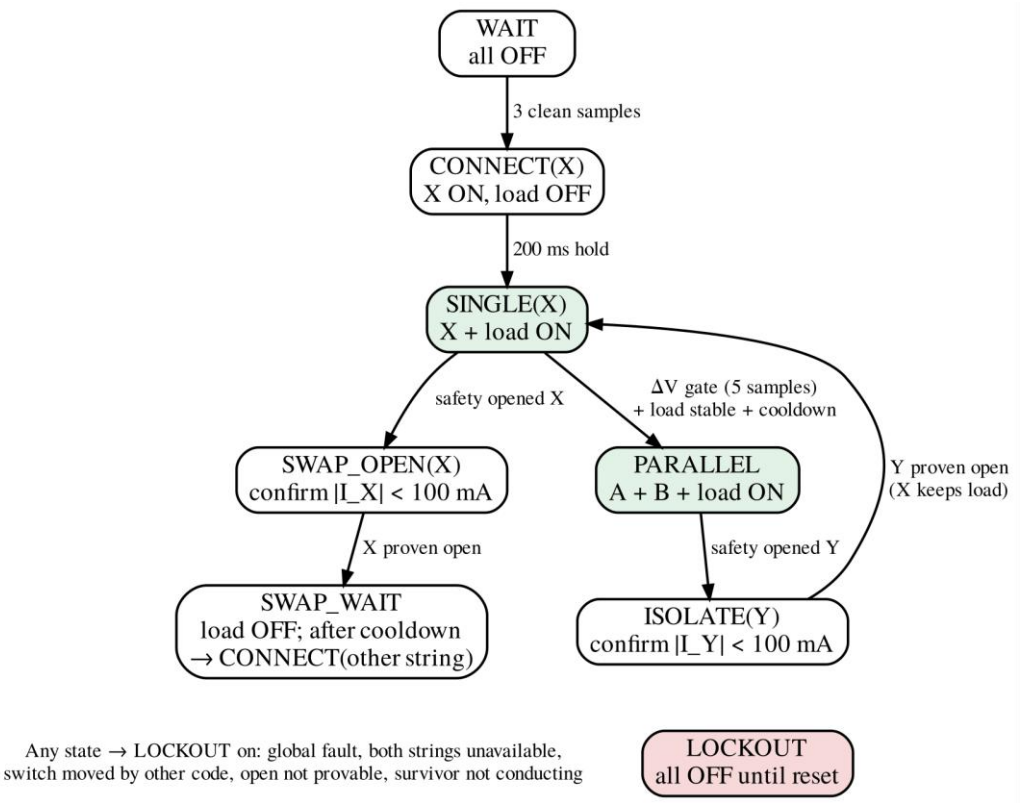


Fig. 3.8: Reconfiguration state machine (X: connected string, Y: other string)

After start-up the module waits until the safety layer reports ready, then closes one string with the load switch still open and holds this state for 200 ms. The hold is longer than the 160 ms the safety layer needs to judge a freshly connected string (a 100 ms settling interval plus three samples), so a string that is out of limits is rejected before it reaches the motors. The load switch then closes. Paralleling is compiled in only when a bench test has confirmed that each sensor reads its own string while that string is disconnected; otherwise the measured voltage difference would be meaningless, and the firmware runs on one string at a time. Table 3.7 lists the five guarantees of the module.

Table 3.7: Guarantees of the reconfiguration module

	Guarantee	**Enforced by**
G1	Strings A and B are commanded on together only after the ΔV gate passed on the samples immediately before the close	Gate check before the close; paralleling compiled in only when the sensors read string-side voltage
G2	Break-before-make: the old string is proven open by current readback before a new string closes	Open confirmation (three samples below 100 mA)
G3	A switch that the safety layer blocks is never closed	Block-mask check before and inside the close routine
G4	At least 500 ms between topology changes, including openings forced by the safety layer	Cooldown timer restarted on every change
G5	Any state the module cannot explain leads to all-off until reset	Command/output-register reconciliation; LOCKOUT

3.3.5 Paralleling gate

While string X is connected and carries current I_X, and string Y is open, both sensors read string-side terminal voltages. The bus voltage is $V_{bus} = V_X - I_X R_{path}$, where R_{path} is the unknown resistance between X's sense point and the bus. The voltage that will drive the current when Y is closed is

$$\Delta V_{real} = V_Y - V_{bus} = \Delta V_{lo} + I_X R_{path} \tag{3.8}$$

where $\Delta V_{lo} = V_Y - V_X$. Because ΔV_{real} is linear in R_{path} and $0 \le R_{path} \le R_{max}$, its magnitude can never exceed the larger of $|\Delta V_{lo}|$ and $|\Delta V_{hi}|$, where $\Delta V_{hi} = \Delta V_{lo} + I_X R_{max}$. The gate therefore requires both bounds to be below 100 mV on five consecutive samples (100 ms), with the load current stable (a change of less than 200 mA between samples) and the cooldown expired. This guarantees a true difference below 100 mV without knowing R_{path} exactly.

R_{max} = 50 mΩ is the sum of two 20 mΩ MOSFETs at their datasheet maximum, the 2 mΩ shunt, and an 8 mΩ allowance for wiring and for the higher on-resistance at low gate drive; the allowance is an assumption still to be confirmed by measurement. Fig. 3.9 shows the consequence: the admissible window narrows as the load current rises, is (−100, 0) mV at 2 A and closes completely at 4 A. Under heavy load the system stays on one string and joins the second when the load falls.

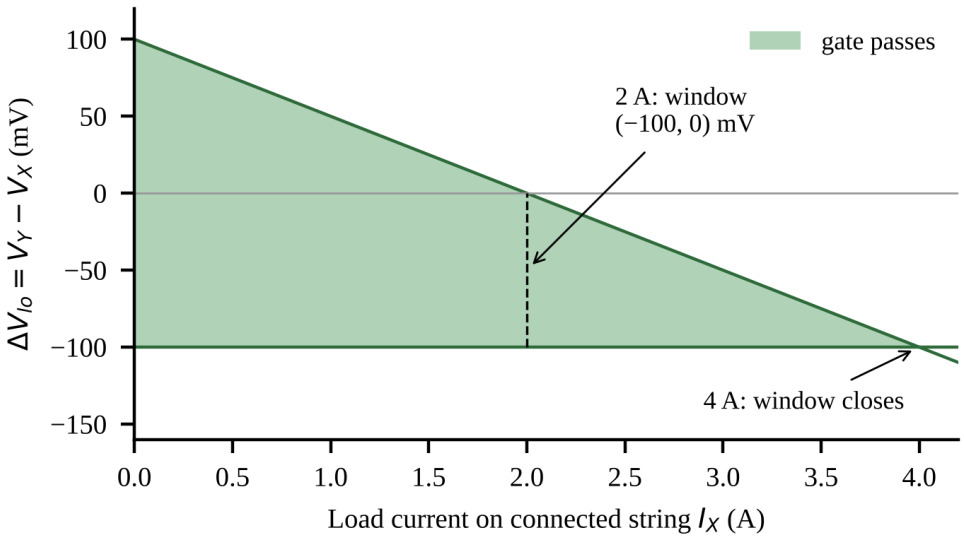


Fig. 3.9: Admissible region of the paralleling gate for a maximum path resistance of 50 mΩ (calculated)

Within the gate, the current that flows when the strings are joined is bounded by the loop resistance of both strings and both pairs:

$$I_{join} \le \Delta V / R_{loop} < 0.1\ \text{V} / R_{loop} \tag{3.9}$$

Using the datasheet impedance limit of 26 mΩ per cell (measured at 1 kHz) [1] as an estimate, four MOSFETs at 20 mΩ and about 30 mΩ of wiring, $R_{\text{loop}} \approx 266$ mΩ and $I_{\text{join}} \leq 0.38$ A. Counting only the four MOSFETs (80 mΩ) gives a pessimistic bound of 1.25 A. Both are an order of magnitude below the 12 A trip; they are calculated values and have not yet been measured. Because the board has no precharge path, the controller always starts on the string with the higher voltage: only then can the load pull that string down towards the idle one and close the gap.

3.3.6 Isolation and swap sequences

Isolation while paralleled. If the safety layer opens string Y while both strings are connected, no switch has to close, because string X is already carrying the load. The module enters ISOLATE, requires the current in Y to fall below 100 mA on three consecutive samples, and checks that X is actually carrying the load. If Y is not confirmed open within 300 ms, or if X carries no current although the load drew at least 1.5 A before the trip, the module enters LOCKOUT.

Swap from a single string. If the only connected string X is opened, the module keeps the motors commanded while it proves X open. This ordering matters: in host simulation, stopping the motors first made a stuck pair read almost zero current, which looked like a successful opening, and the other string was then closed onto it. The proof also requires that X carried at least 300 mA just before the trip, because zero current afterwards proves nothing if no current flowed before. Only then is the load switch opened, the cooldown and a 20 ms dead time observed, and the other string connected through the CONNECT hold.

Faults never clear in software, and a string that has failed to conduct is not retried until reset. This bounds the number of topology changes per boot and, together with the 500 ms cooldown, prevents oscillation between topologies.

3.3.7 Operator interface and motor gating

The operator controls the rover through the Nucleo user button and the serial terminal (Table 3.8). The motors may run only while a string is connected to the load; the motor gate in the control loop stops them whenever the reconfiguration module releases the load. In the parallel build, isolating one string does not release the load, so the rover keeps driving. In the single-string build, a swap passes through a period with the load switch open, after which the motors stay stopped until the operator presses the drive button again; the rover never restarts by itself.

Table 3.8: Operator inputs

Input	Action
Drive button (PC13)	Start or stop the motors, only while a string feeds the load
‘0’ on the serial terminal	Emergency stop: all switches off, LOCKOUT until reset
‘f’ then ‘a’ or ‘b’	Inject an over-current on string A or B (demonstration hook)
Reset button	Clear all latched faults and restart; all switches start off

3.3.8 Display and telemetry

The OLED shows eight rows: the voltage, current and sensor status of each string, the voltage difference, the topology and whether it is confirmed by current readback, the switch and motor states, the fault label, the per-bus failure and recovery counters, and the loop count with the uptime. To keep the shared I²C bus available for sensor A, only one display page is written per control cycle. A CSV line is sent over the ST-Link virtual COM port (USART2, 115 200 baud) five times per second, and event lines record each trip, topology change and lockout reason.

3.3.9 Failure modes and effects analysis

Table 3.9 summarises the failure modes analysed for the switching hardware and the control code, following the structure of ISO 26262 and IEC 61508 analyses [17, 18]: what fails, how it is detected, and the safe response. Two entries cannot be detected by the controller and are mitigated only by hardware: a pair shorted before power-up with no load current flowing, and inrush into the motor driver’s input capacitance when the load switch closes, which lasts far less than one sampling period.

Table 3.9: Failure modes and effects analysis of the switching hardware and control code

Failure	Detection	Safe response
Close onto a string at a large ΔV	ΔV gate on both bounds, five samples	Join refused; stay on one string
Sensor reads the bus instead of its string	Not detectable in firmware	Paralleling compiled out unless confirmed on the bench
Pair stuck closed during a swap	Open never confirmed; STUCK at about 160 ms	LOCKOUT; new string never closed
Open cannot be proven (no load before the trip)	Current before the trip < 300 mA	LOCKOUT
Joined pair fails open or is intermittent	< 100 mA while the total load ≥ 1.5 A	Open it; no rejoin until reset
Other code moves a switch	Command differs from output register	LOCKOUT
NPN shorted collector to emitter	Current in an OFF string	STUCK: all off including load
NPN open	Pair does not conduct when commanded	String unavailable; other string used
MOSFET gate oxide punctured	Readings contradict the commanded state	All off
Sensor lost or bus held low	Timeout; recovery fails	Sensor fault: all off, latch
Loop hangs	Watchdog timeout	Reset into hardware all-off
HAL fault	Error handler entered	Direct register write: all off
Pair shorted before power-up, no load	Not detectable	Fuse; bench procedure
Load-switch inrush into the driver’s input capacitance	Not visible at 20 ms sampling	Fuse; precharge in the next revision

3.4 Project Timeline

Table 3.10 lists the project activities in the order in which they were carried out. The dates are to be filled in by the project team.

Table 3.10: Project timeline (to be completed by the project team)

Sr.	Activity	Start	End	Duration	Status
1	Literature survey and component selection				
2	Simulation study (MATLAB/Simulink)				
3	Switching prototype (breadboard and perfboard)				
4	Sensing and display prototype				
5	PCB design and fabrication				
6	Drivers, sensing, display and telemetry firmware				
7	Safety layer (firmware and tests)				
8	Reconfiguration state machine (firmware and tests)				
9	Rover integration and fault-isolation trials				
10	Research paper				
11	Project report				

This chapter described the block diagram of the system, the hardware from the cells to the rover, and the firmware that controls it, including the safety layer, the reconfiguration state machine and the failure analysis. Chapter 4 now examines how each part was tested and what the results show.

Chapter 4
Results and Analysis

Testing addressed three questions: whether the firmware makes the right decisions in every situation, whether the hardware behaves as calculated, and whether the complete system isolates a faulted string on the moving rover. This chapter presents the testing approach and standards followed, the simulation results, the bench measurements, the rover trials, the design errors found along the way, and a discussion of the results.

4.1 Testing Approach and Standards Followed

Results are reported at three evidence levels that are never mixed: *calculated* values come from equations with stated inputs; *simulated* values come from running the unmodified firmware modules on a computer against a model of the pack; *measured* or *observed* values come from the fabricated hardware. Testing proceeded from logic to hardware: the decision logic was first exercised in simulation, then on the board powered by USB without cells, then with cells on the bench, and finally on the rover. Table 4.1 maps the standards and specifications used to the part of the project where each applies.

Table 4.1: Mapping of standards and specifications to the project

Standard / specification	Scope	Where it applies
ISO 26262 [17], IEC 61508 [18]	Functional safety	Safe state, fault latching, watchdog, FMEA structure (concepts only; no compliance claimed)
ISO/IEC 9899:1999 [20]	C programming language	Firmware and host simulation compiled as C99 with strict warnings
NXP UM10204 [19]	I²C bus	Sensor and display buses; nine-pulse bus clear
Cell specification [1]	Cell limits	Voltage and current thresholds of the safety layer
Component datasheets [12, 13, 14, 15, 16]	Component ratings	Gate-drive calculations, sensing, watchdog timing, supply limits

4.2 Host Simulation

The safety module and the reconfiguration module were compiled on a computer with strict warning flags (gcc, C99 [20], -Wall -Wextra -Wpedantic -Wconversion -Wshadow) without modification, producing no warnings, and were run against a model of the pack: open-circuit voltage and internal resistance for each string, a 30 mΩ switch pair, a current-sink load, injected faults (stuck pair, failed-open pair, lost sensor) and measurement noise of ±3 mV and ±5 mA. A stub that replaces the switch driver checks guarantees G1 to G4

on every switch-on command, using the model's true voltage difference rather than the sensed one.

The simulation verifies decision logic and sequencing only. It contains no hardware abstraction layer, no sensor conversion or I²C timing, no switching transients, wiring inductance or capacitor inrush, and its resistances are assumptions. All results in this section are simulated.

Table 4.2 summarises the safety-layer scenarios; all 22 passed. Table 4.3 lists the 18 reconfiguration scenarios, run in the parallel build (12) and the single-string build (6); all passed. In the parallel demonstration scenario (P2), string B opened 60 ms after the loop that received the injected fault, its open state was confirmed at 120 ms, the load was never interrupted and string A carried the load (simulated timing).

Table 4.2: Safety-layer host-simulation scenarios (simulated)

Scenario	**Result**
Boot hold-off	All switches blocked until the third clean sample
Injected over-current on B while A runs	B trips on the third sample (60 ms); A and the load keep running; label shows the fault as injected
Faulted string re-closed by other code	Forced off in the next cycle
Two 12.5 A samples, then a normal one	No trip; three consecutive samples trip
Low voltage on a freshly connected string	UV latched 140 ms after the low reading (100 ms settle + 3 samples)
0 V on all-off strings for 20 samples	No fault (no start-up deadlock)
600 mA through an OFF string	STUCK; all switches off including the load
Current within 100 ms of an OFF command	Not treated as stuck
Sensor B unreadable	All off in the same cycle; everything blocked
−4.5 A charge current; 12.65 V	OCC and OV latched
Sensor missing at start-up	Everything blocked

Table 4.3: Reconfiguration host-simulation scenarios (simulated)

ID	**Scenario and outcome**
P1	Start-up and join at rest; true ΔV at the join 37.5 mV
P2	Parallel demonstration: B opens, load never interrupted, motors keep running, A carries the load
P3	ΔV of 150 mV: never joins
P4	0.8 A load pulls A into the gate window; joins while driving (true ΔV 68.7 mV)
P4b	2 A load: no join while driving; joins at rest
P5a / P5b	95 mV joins; 105 mV refused
P6	Failed-open B dropped and never rejoined
P8	Switch turned on by other code: LOCKOUT

ID	Scenario and outcome
P9	Sensor lost: LOCKOUT
P10	Both strings trip: LOCKOUT
P11	Single-string swap in the parallel build
S1	Single-string build: start-up on B
S2	Single-string build: demonstration swap from B to A with break-before-make
S3	Fault with motors stopped: LOCKOUT (open cannot be proven)
S4	Stuck B: STUCK, then LOCKOUT; A never closed
S6	Emergency stop command: LOCKOUT
S7	Idle string faults (ignored), then both down: LOCKOUT

P1 to P11 (12 scenarios, counting P5a and P5b separately) use the parallel build; S1 to S7 (6 scenarios) use the single-string build. All 18 passed.

Fig. 4.1 shows the simulated switching sequence for an injected fault on string B in the two builds. In the parallel build, string A and the load switch never change state; only string B opens. In the single-string build, the load switch opens once string B is confirmed open, string A closes 500 ms after the trip because of the cooldown, and the load switch closes after the 200 ms CONNECT hold. String A and string B are never on together. The times are measured from the loop in which the injection command is received; a further 0 ms to 20 ms elapse between the keystroke and that loop.

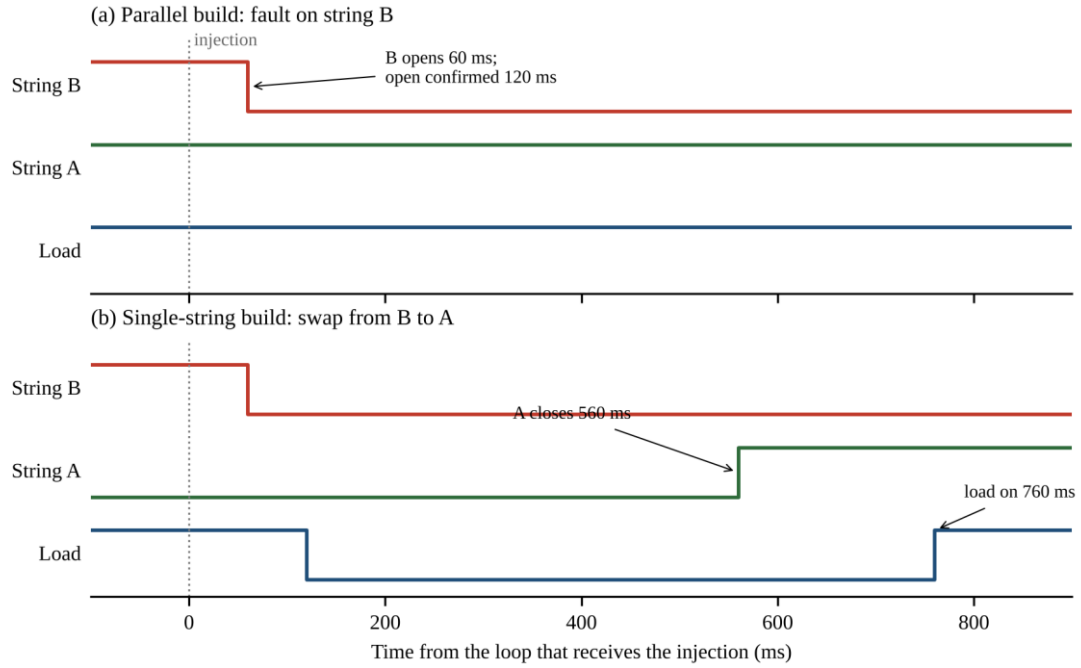


Fig. 4.1: Simulated switching sequence after an injected fault on string B

Fuzzing ran 400 randomized runs of 30 s for each build, with random start voltages (10.6 V to 11.8 V), motor on/off events and currents (0.3 A to 3.3 A), fault injections, and failed-open or stuck pairs. No invariant was violated in any of the 800 runs.

Mutation testing checks whether the test harness would actually notice a bug. Deliberate errors were introduced into the reconfiguration module one at a time (Table 4.4). Six of the seven single mutants were detected. The undetected one removed one of two redundant block-mask checks; every caller performs the first check, so the second is defence in depth, and a mutant that removed both checks was detected.

Table 4.4: Mutation tests on the reconfiguration module (simulated)

Deliberate error	**Detected?**
ΔV gate raised from 100 mV to 300 mV	Yes
Gate on ΔV_{lo} only (load term removed)	Yes
No cooldown before a join	Yes
Swap without the ‘current flowed before’ proof	Yes
Swap without waiting for open confirmation	Yes
Motors stopped before the open proof	Yes
Block-mask check removed inside the close routine only	No (redundant check)
Block-mask check removed in both places	Yes

4.3 Switch Stage and Sensing on Hardware

Table 4.5 compares bench measurements with the calculated values. The measured gate-source voltage of −10.8 V on the breadboard prototype is 0.8 V smaller in magnitude than the calculated −11.6 V of (3.3), consistent with the pack sitting below the 11.9 V assumed and with the assumed junction drops; it remains well beyond the −4.0 V worst-case threshold. With the pair open, the load side read about 11 mV on the breadboard prototype, which is the floor of the multimeter in that range. In a blocking test on the pair, the bus side read 0.9 mV with the string energised, less than 0.01% of the string voltage.

Table 4.5: Bench measurements compared with calculation

Quantity	**Measured**	**Calculated**	**Set-up**
V_{GS}, channel on	−10.8 V	−11.6 V	Prototype
V_{GS}, channel off	0 V	0 V	Prototype
Load-side voltage, channel off	≈ 11 mV	—	Prototype
NPN base voltage, channel on	≈ 0.7 V	0.7 V	Prototype
Bus side, pair open	0.9 mV	—	Pair
String A voltage, all switch states	11.95–11.96 V	—	PCB
String B voltage, all switch states	11.86–11.87 V	—	PCB
Voltage difference between strings	90 mV	< 100 mV (gate)	PCB
I²C failed reads / bus recoveries	0 / 0	—	PCB
Loops at uptime 00:01:11	3485	3475–3525	PCB

The loop-count prediction assumes 20 ms per loop, a 1.5 s boot screen and an uptime display truncated to whole seconds.

On the fabricated board, an I²C scan found the OLED (0x3C) and sensor A (0x40) on I2C1 and sensor B (0x40) on I2C3, and both sensors returned the expected manufacturer identifier 0x5449 [14]. The firmware then cycled the switches through all-off, string A alone, string A with load, all-off, string B alone and string B with load; the log of this test is reproduced in Appendix A. String A read 11.95 V to 11.96 V and string B 11.86 V to 11.87 V in every state, so neither reading depended on the switch state. This confirms that each sensor reads its own string, the precondition for paralleling. The difference between the strings was 90 mV, inside the 100 mV gate. The failed-read and bus-recovery counters stayed at zero on both buses through every transition, including closing and opening the load switch, which was a real risk because the I²C lines run close to the power path.

Fig. 4.2 shows the bench set-up with cells and the OLED during one run. The display shows a loop count of 3485 at an uptime of 1 min 11 s. Since 3485 × 20 ms = 69.7 s and the uptime is shown in whole seconds, the difference of 1.3 s to 2.3 s is consistent with the boot screen that precedes the control loop, and with a 20 ms loop period in this run.

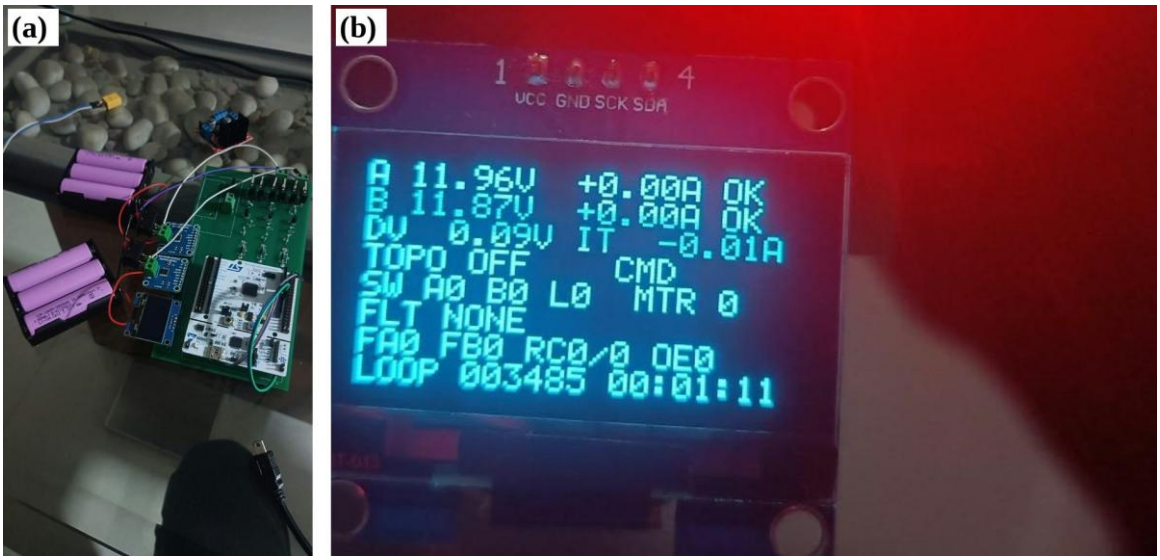


Fig. 4.2: Bench test: (a) both strings on the PCB, (b) OLED status at uptime 00:01:11

4.4 Safety Layer on Hardware

Table 4.6 lists the checks performed on the board powered by USB with no cells connected, and with the reset button held; all passed.

Table 4.6: Safety-layer checks on the board (measured / observed)

Check	Observed
Start-up	Reset cause printed, both sensors found, safety ready reported
Injected over-current on string B	Fault latched and shown as injected
Re-close faulted string B	Command rejected
Close string A with no cells	UV latched on A (0 V < 7.50 V), as designed
Reset button	All faults cleared
Sensor B data line disconnected	Sensor-lost fault; stayed latched after reconnection until reset
Reset button held	String A drive circuit below the NPN turn-on level; string A off

Board powered from USB with no cells connected, except for the last row.

4.5 Fault Isolation on the Rover

The central test used the parallel build on the rover, with both strings connected and the rover driving, and injected an over-current on one string through the fault-injection hook. Table 4.7 gives the results. In 20 of 20 trials with the fault on string B, and 20 of 20 trials with the fault on string A, the faulted string opened and the rover continued to drive on the remaining string. Because the controller is powered from net STR_A upstream of the string A switch, opening string A does not remove controller power. When faults were injected on both strings, both opened and the system ended in the all-off state, as required by the rule that a pack with no available string is shut down.

Table 4.7: Fault isolation on the rover (observed)

Injected fault	Trials passed	Observed outcome
Over-current, string B	20 / 20	B opened; rover kept driving on A
Over-current, string A	20 / 20	A opened; rover kept driving on B
Both strings	—	Both opened; system all-off

Parallel build; both strings connected and the rover driving at the moment of injection. Outcomes observed by the operator; timing not recorded.

Each trial followed the same procedure. The firmware was built with paralleling enabled after the bench check had confirmed that each sensor reads its own string while disconnected. After a reset, the system connected the higher string, closed the load switch and joined the second string once the paralleling gate was satisfied. The operator started the motors with the drive button, injected the fault from the serial terminal while the rover was moving, observed whether the faulted string opened and whether the rover kept driving, and then pressed reset to clear the latched fault before the next trial.

These outcomes were observed by the operator. Trip latency, the transfer of current to the remaining string and the absence of a gap at the load switch were not recorded with an oscilloscope or a logged trace in these trials, so the results demonstrate the functional behaviour of the hardware and firmware together, not their timing.

Table 4.8 lists the bench tests that have been designed but not yet performed. Their results are not claimed anywhere in this report.

Table 4.8: Bench tests designed but not yet performed

Test	**Purpose**
Trip latency on an oscilloscope	Confirm the calculated 60 ms to 80 ms from fault to switch opening
Watchdog hang test	Force a hang and confirm a reset within the calculated 0.34 s to 0.94 s
Real under-voltage trip	Lower a bench supply through 7.5 V and record the trip voltage
Stuck-switch test with a fused jumper	Bypass a pair and confirm STUCK, load-switch opening and LOCKOUT
Join current	Measure the current when the second string joins, against the bound of (3.9)
Reverse blocking of a pair	Confirm blocking with the bus above an open string

4.6 Engineering Corrections

Thirteen design errors were found and corrected during development (Table 4.9). They are recorded because the way they were found is itself a result: several were caught only because a specific check was run, and those checks became part of the method. In addition, the host simulation exposed the swap-ordering defect described in Section 3.3.6 before any hardware test.

Table 4.9: Design errors found and corrected

#	Error	Found by	Cost
1	PA2/PA3 assigned to both MOSFET control and USART2	Pin-map review against the board manual	None, caught on paper
2	PA8/PA9 assigned as analogue inputs; neither pin has one on this device	Datasheet check	None, caught on paper
3	Load switch required by the design but missing from the pin map	Cross-check of plan against pin map	None, caught on paper
4	Switch stage powered from 3.3 V; gate drive insufficient	Bench: no switching	Several hours
5	MOSFET sources tied to the battery rail, bypassing one device	Bench, then netlist review	Two MOSFETs, several hours
6	Gate pull-up in series between collector and gate	Netlist review	Rework
7	Gate pull-up returned to the rail instead of the source node	Netlist review	Rework
8	Both string terminals on one net, shorting the strings before their switches	Netlist review	Rework
9	Both INA260 modules fixed at 0x40	Module inspection	Two extra bus pins
10	Error value −9999 collides with a valid −10 A reading	Code review	None, caught before use
11	NPN destroyed while probing with 11 V present	Bench	One transistor
12	PB3 and PB4 are debug pins at reset; outputs silently inactive	Bench: apparent hardware fault	Debugging time
13	Motor IN3/IN4 assignments in firmware contradicted the netlist	Netlist cross-check	None, caught before test

Three devices were lost during breadboard work. Two IRF4905 MOSFETs read 2.3 Ω from gate to source instead of a megaohm-range value, indicating punctured gate oxide, and one NPN failed while a multimeter probe was applied to its base with 11 V present on the board. The working practice was changed in response: all grounds are connected before power is applied, resistor values are checked out of circuit, and clipped probes are used near live power rails.

4.7 Discussion of Results

Table 4.10 summarises the status of each subsystem and the evidence behind it. The results show that string-level isolation can be achieved with discrete parts and a simple control structure. The separation of authority is the most important design decision: the safety layer can only remove connections and the reconfiguration module can only add them under proven conditions, so an error in the reconfiguration logic cannot override a protective

action. In the parallel case, isolating a string requires no closing action at all, which is why the load stayed powered in every rover trial.

Table 4.10: Status of each subsystem and the evidence behind it

Subsystem	**Status**	**Evidence**
Switch channels (three pairs)	Built; all actuate	Measured
Hardware default-off	Built; reset test passed	Measured
String voltage sensing	Working; 0 bus errors	Measured
Safety layer	Implemented	Simulated (22/22); on-board checks
Reconfiguration state machine	Implemented	Simulated (18/18, 800 runs, mutation tests)
Fault isolation on the rover	Working	Observed (40/40)
Watchdog	Implemented	Timeout calculated; hang test not yet run
Trip latency, join current	Not measured	Calculated / simulated only
Precharge, per-cell and temperature sensing	Not implemented	—

The main gaps in the evidence are timing and current waveforms. The 60 ms to 80 ms trip time, the join current bound of (3.9) and the open-confirmation timing are calculated or simulated, not measured, and the path-resistance allowance in the paralleling gate is an assumption. Measuring these with an oscilloscope is the first item of future work.

This chapter presented the testing approach, the simulation results, the bench measurements, the rover trials and the corrections made during development. Chapter 5 now discusses the advantages, limitations, applications and cost of the system.

Chapter 5
Advantages, Limitations and Applications

A fair assessment of a prototype covers what it does well, what it does not do, where it could be used and what it costs. This chapter covers each in turn and relates the work to the Sustainable Development Goals.

5.1 Advantages

- **Keeps running after a string fault.** A faulted string is removed while the other string continues to supply the load, demonstrated in 40 rover trials.
- **Blocks current in both directions.** The back-to-back pair isolates a string whether the bus is above or below it.
- **Safe by default.** Hardware pull-downs open every switch whenever the controller is in reset, unpowered or hung; the watchdog turns a hung loop into a reset.
- **Never parallels blindly.** Strings are joined only after a gate that bounds the true voltage difference below 100 mV under load.
- **Proven break-before-make.** A switch is considered open only when its current confirms it.
- **Auditable control.** Rule-based logic, a fixed 20 ms loop and a single module that can close switches make every action traceable to a rule.
- **Low cost and reproducible.** The switching uses standard discrete parts and a general-purpose development board.

5.2 Limitations

- **String-level sensing only.** The INA260 measures the sum of three cells. A single cell can exceed 4.20 V or fall below 2.50 V while the string sum stays within limits.
- **No temperature sensing.** Thermal faults, a primary lithium-ion hazard, cannot be detected by this hardware.
- **No precharge path.** Paralleling relies on the voltage gate alone, and closing the load switch charges the motor driver’s input capacitance through the path resistance only. A fuse in each string is required.
- **Switch heating near the trip current.** Without heatsinks, a sustained current of about 11 A or more would overheat the MOSFETs before the 12 A trip acts (Section 3.2.4).

- **Firmware cannot stop a short circuit.** The 60 ms to 80 ms trip time protects against sustained overload only.
- **Over-current demonstrated by injection.** The rover cannot approach the 12 A trip, so over-current faults were emulated; injection exercises the real decision-to-switching path but not the sensor's response to a real over-current.
- **Timing not measured.** Trip latency, join current and dead time were not measured with an oscilloscope.
- **Controller supply on string A.** String A is always slightly discharged by the controller, sensor A also carries the controller's supply current, and a full string (12.6 V) exceeds the 12 V maximum VIN of the Nucleo board [16], which restricts string A to 12.0 V or less.
- **Coarse granularity.** One bad cell removes its whole string, which in a 3S2P pack is half of the capacity. This is the deliberate trade of string-level switching: fewer switches and lower conduction loss at the cost of coarser control.
- **Not certified.** The system applies functional-safety concepts but has not been assessed against ISO 26262 or IEC 61508.

5.3 Applications

- Mobile robots and automated guided vehicles that must keep moving after a battery fault.
- Small electric vehicles and drones with two or more parallel strings, where mission continuity matters more than full capacity.
- Backup and stationary storage built from parallel strings, where a faulted string should be removed without taking the system offline.
- Teaching and research platforms for battery management, functional-safety concepts and embedded control, since every part is discrete and observable.

5.4 Cost of the System

Table 5.1 lists the cost of the components fitted to the rover system, using the unit prices recorded in the project bill of materials. Several parts used in the final build were selected after that bill of materials was prepared (the INA260 modules, the 1N5400 diodes, the L298N module and the tracked chassis) and have no recorded price; their cost is to be added by the project team. The largest recorded share is the Nucleo development board, followed by the cells.

Table 5.1: Cost of the components (prices from the project bill of materials)

Sr.	Component	Qty.	Unit price (₹)	Amount (₹)
1	Li-ion cell, Samsung INR18650-30Q	6	250	1,500
2	P-channel MOSFET IRF4905	6	50	300
3	NPN transistor 2N2222A	3	5	15
4	Resistor 1 kΩ	3	1	3
5	Resistor 10 kΩ	3	1	3
6	Resistor 100 kΩ	3	1	3
7	STM32 Nucleo-F446RE (Nucleo-64)	1	1,600	1,600
8	OLED display 128×64, I²C	1	150	150
9	Custom two-layer PCB (set of 5)	1	500	500
10	Zero PCB (perfboard) for prototypes	2	40	80
11	Breadboard (830-point)	1	80	80
12	Signal wire, 22 AWG	3	8	24
13	Header pins (male and female strips)	4	15	60
14	Consumables (solder, flux, heat-shrink, tape, ties)	1	200	200
	Total of priced components			**4,518**
15	INA260 breakout module (×2)		To be added	To be added
16	1N5400 diode (×3)		To be added	To be added
17	L298N motor-driver module		To be added	To be added
18	Tracked chassis, two DC motors and kill switch		To be added	To be added

5.5 Mapping to Sustainable Development Goals

The United Nations 2030 Agenda defines 17 Sustainable Development Goals (SDGs) [21]. Table 5.2 maps the project to the goals it contributes to most directly.

Table 5.2: Mapping of the project to the Sustainable Development Goals

SDG	Contribution
SDG 7: Affordable and clean energy	Safer use of lithium-ion storage: a faulted string is removed while the remaining capacity stays in service
SDG 9: Industry, innovation and infrastructure	A low-cost, reproducible switching and protection design built from standard discrete parts
SDG 12: Responsible consumption and production	Isolating one faulted string instead of disabling the whole pack keeps the healthy cells in use

This chapter summarised the strengths of the system, its limitations, its possible applications, its cost and its contribution to the Sustainable Development Goals. The final section draws the conclusions of the work and outlines what remains to be done.

Conclusion and Future Scope

This project set out to make a two-string lithium-ion pack on a mobile rover survive the failure of one string. A string-level reconfigurable battery management system was designed, built on a two-layer printed circuit board and tested on a tracked rover powered by a 3S2P pack of Samsung INR18650-30Q cells.

Each string and the load are switched by two IRF4905 P-channel MOSFETs connected back to back with their sources joined only to each other, so that an open pair blocks current in both directions. A discrete NPN level shifter drives the gates to a calculated −11.6 V (−10.8 V measured on the prototype), and base pull-down resistors keep every switch open whenever the controller is not actively driving it. Two INA260 monitors on separate I²C buses measure each string.

The STM32F446RE firmware runs a fixed 20 ms loop. A latching safety layer enforces voltage, current, sensor and stuck-switch limits and may only open switches; a separate reconfiguration state machine is the only code that may close one, under five guarantees that cover the paralleling gate, break-before-make, safety blocking, a 500 ms cooldown and lockout on any unexplained state. The firmware passed all host-simulated scenarios and 800 randomized runs without violating a safety invariant, and the test harness detected the deliberately injected code errors. On the rover, an injected over-current isolated the affected string while the other kept the vehicle driving in all 40 trials.

Beyond the prototype itself, the work showed how much of battery safety lies in details that a simple functional test does not reveal: the direction of a body diode, the order in which motors are stopped, and the difference between a sensor that reads its string and one that reads the bus. Several of these were found only by reading the netlist or by running the firmware against a model.

Future Scope

- Add a precharge branch and a separate precharge switch to the next board revision, so that strings and the load can be connected through a current-limiting resistor.
- Strengthen the base pull-down on the string A channel (2.2 kΩ) or move it to a pin without a debug function, and power the controller from a supply independent of string A.
- Add per-cell voltage measurement and temperature sensing so that cell-level faults and thermal events can be detected.

- Measure trip latency, join current and the load-switch inrush with an oscilloscope, and record logged traces of the rover trials.
- Add per-string state-of-charge estimation so that string alternation and charge balancing can be driven by measured state rather than by faults alone.
- Extend the design to more parallel strings, where isolating one string costs a smaller share of the capacity.

References

[1] Samsung SDI Co., Ltd., “Specification of product: Lithium-ion rechargeable cell for power tools, model INR18650-30Q,” Version 1.0, Feb. 2015.

[2] W. Han, T. Wik, A. Kersten, G. Dong, and C. Zou, “Next-generation battery management systems: Dynamic reconfiguration,” IEEE Industrial Electronics Magazine, vol. 14, no. 4, pp. 20–31, Dec. 2020, doi: 10.1109/MIE.2020.3002486.

[3] S. Steinhorst et al., “Distributed reconfigurable battery system management architectures,” in Proc. 21st Asia and South Pacific Design Automation Conference (ASP-DAC), 2016, pp. 429–434, doi: 10.1109/ASPDAC.2016.7428049.

[4] A. Škegro, T. Wik, B. Bijlenga, A. Bessman, and C. Zou, “System-level assessment of dynamic reconfiguration for lifetime and cost outcomes in electric vehicle battery packs,” Nature Communications, vol. 17, Art. no. 5980, 2026, doi: 10.1038/s41467-026-74951-8.

[5] F. Tang et al., “Reconfigurable battery system topology, operation and feature: A comprehensive survey,” Journal of Energy Storage, vol. 168, Art. no. 122573, 2026.

[6] Z. M. Pinter, D. Papageorgiou, G. Rohde, M. Marinelli, and C. Træholt, “Review of control algorithms for reconfigurable battery systems with an industrial example,” in Proc. 56th International Universities Power Engineering Conference (UPEC), 2021, doi: 10.1109/UPEC50034.2021.9548259.

[7] T. Morstyn, M. Momayyezan, B. Hredzak, and V. G. Agelidis, “Distributed control for state-of-charge balancing between the modules of a reconfigurable battery energy storage system,” IEEE Transactions on Power Electronics, vol. 31, no. 11, pp. 7986–7995, Nov. 2016.

[8] M. Lemeon and J. Regash, “Development of a reconfigurable battery pack architecture for fault isolation and lifetime extension,” Transactions on Energy Storage Systems and Innovation, vol. 2, no. 1, pp. 52–58, 2026, doi: 10.17051/T-ESSI/02.01.06.

[9] H. M. O. Canilang, A. C. Caliwag, and W. Lim, “Novel battery management systems: Enhancing flexibility and fault tolerance,” Journal of Energy Storage, vol. 107, Art. no. 114898, 2025.

[10] Z. Gao, C. S. Chin, J. H. K. Chiew, J. Jia, and C. Zhang, “Design and implementation of a smart lithium-ion battery system with real-time fault diagnosis capability for electric vehicles,” Energies, vol. 10, no. 10, Art. no. 1503, 2017, doi: 10.3390/en10101503.

[11] Q. Zhao, F. Tang, and B. Zhang, “Adaptive SOC estimation of reconfigurable battery modules based on a hybrid deep learning model,” Electronics, vol. 15, Art. no. 2208, 2026, doi: 10.3390/electronics15102208.

[12] Infineon Technologies AG, “IRF4905PbF HEXFET power MOSFET,” datasheet, Rev. 2.1, Apr. 2026.

[13] Semiconductor Components Industries (onsemi), “P2N2222A amplifier transistors, NPN silicon,” datasheet P2N2222A/D, Rev. 7, Jan. 2013.

[14] Texas Instruments, “INA260 precision digital current and power monitor with low-drift, precision integrated shunt,” datasheet SBOS656C, Dec. 2016.

[15] STMicroelectronics, “STM32F446xC/E: Arm Cortex-M4 32-bit MCU+FPU,” datasheet DS10693.

[16] STMicroelectronics, “STM32 Nucleo-64 boards (MB1136),” user manual UM1724, Rev. 17.

[17] Road Vehicles — Functional Safety, ISO 26262:2018, International Organization for Standardization, 2018.

[18] Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems, IEC 61508:2010, International Electrotechnical Commission, 2010.

[19] NXP Semiconductors, “I²C-bus specification and user manual,” UM10204, Rev. 7.0, Oct. 2021.

[20] Programming Languages — C, ISO/IEC 9899:1999, International Organization for Standardization, 1999.

[21] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” Resolution A/RES/70/1, Oct. 2015.

Appendix A: Sample Code and Test Log

The flowcharts of the firmware are given in Fig. 3.6 (control loop), Fig. 3.7 (safety layer) and Fig. 3.8 (reconfiguration state machine). Fig. A.1 combines them with the hardware in a single view of energy, measurement and command flow. This appendix also reproduces sample code from the safety layer and the error handler, and the log of the switching test on the fabricated board discussed in Section 4.3. The reconfiguration module is not reproduced, as it contains the project-specific contribution.

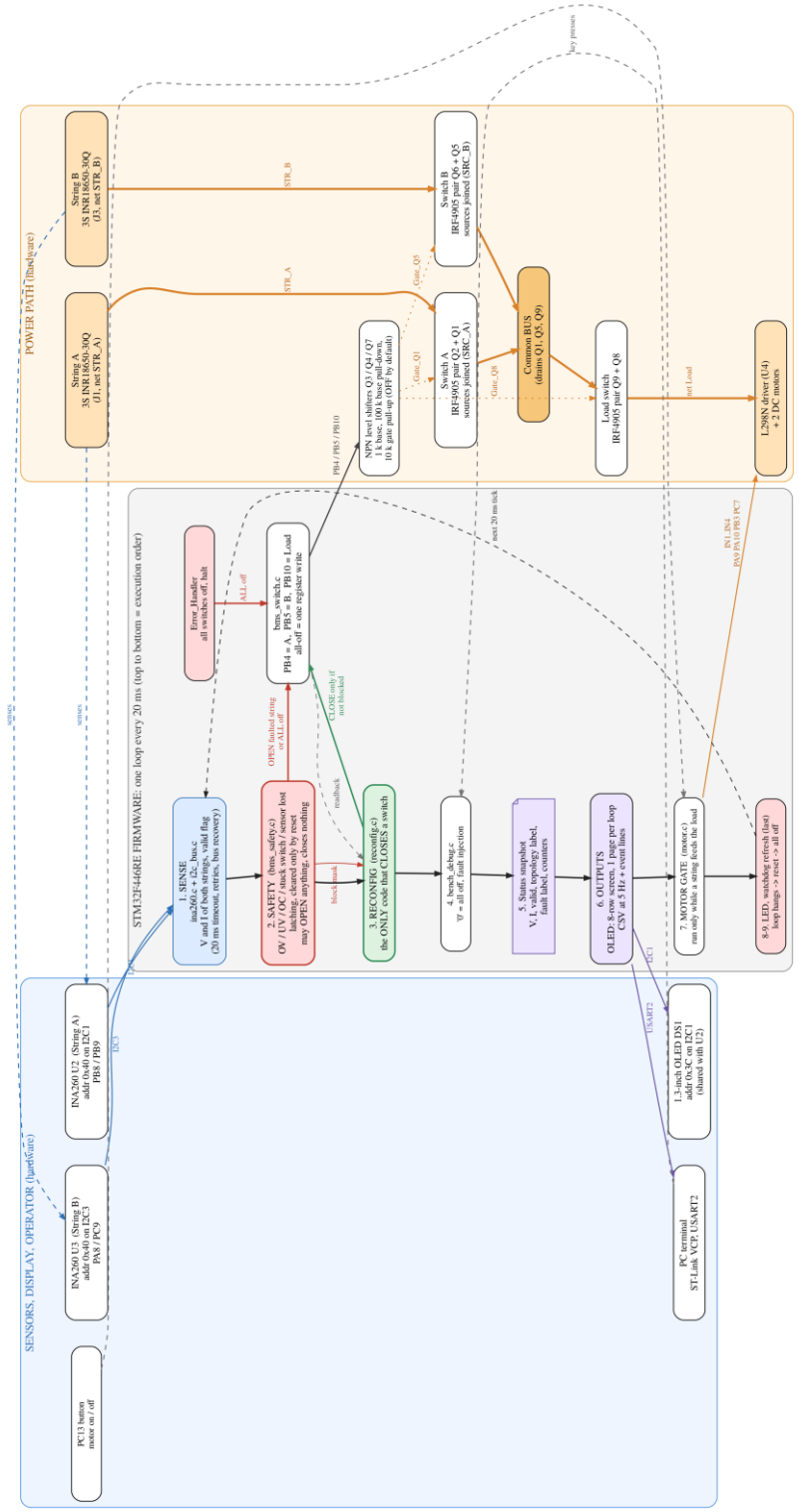


Fig. A.1: Energy, measurement and command flow of the complete system

Listing A.1: Debounce and enforcement in bms_safety.c (excerpt)

```
/* One debounce step. Returns 1 when the check has been bad for N
   consecutive samples. Any good sample resets the count to 0. */
static uint8_t debounce(uint8_t *cnt, uint8_t bad)
{
    if (!bad) { *cnt = 0u; return 0u; }
    if (*cnt < 255u) (*cnt)++;
    return (*cnt >= SAFETY_DEBOUNCE_N) ? 1u : 0u;
}


        uint8_t bad[CHK_COUNT];
        bad[CHK_OV]    = (v_trust && v_mV > SAFETY_OV_MV) ? 1u : 0u;
        bad[CHK_UV]    = (v_trust && v_mV < SAFETY_UV_MV) ? 1u : 0u;
        bad[CHK_OCD]   = ( i_dsg > SAFETY_OC_DSG_MA) ? 1u : 0u;
        bad[CHK_OCC]   = (-i_dsg > SAFETY_OC_CHG_MA) ? 1u : 0u;
        /* Stuck: judged on the REAL current, never the injected one, and
           only once the OFF command has had time to take effect. */
        bad[CHK_STUCK] = (!on
                          && (uint32_t)(now - t_off[s]) >= SAFETY_STUCK_SETTLE_MS
                          && iabs(i_real) > SAFETY_STUCK_MA) ? 1u : 0u;

    /* 6. Enforce, every call. */
    if (any_global())
    {
        Switch_AllOff();                    /* strings AND load switch */
    }
    else
    {
        for (uint8_t s = 0; s < BMS_NUM_STR; s++)
            if (str_bits(s)) Switch_Set(sw_of[s], SW_OFF);
    }
```

Listing A.2: Error handler forcing the safe state

```
void Error_Handler(void)
{
    /* Safe state FIRST, before interrupts go away and before anything
       that depends on HAL - HAL may be the failed subsystem.
       Upper 16 bits of BSRR = atomic reset. */
    GPIOB->BSRR = (uint32_t)(GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_10) << 16u;
    __disable_irq();
    while (1) { }
}
```

In the log below, *rec* and *fail* are the bus-recovery and failed-read counters of I2C1 and I2C3. The current readings of −1 mA to −3 mA are the sensor offset: the INA260 current terminals had not yet been connected in series with the strings when this test was run, so the log demonstrates switching and voltage sensing only.

Listing A.3: UART log of the bus scan and switching test on the fabricated board

```
Scanning I2C1 (PB8/PB9):
  found 0x3C <- SSD1306 OLED
  found 0x40 <- INA260 String A
Scanning I2C3 (PA8/PC9):
  found 0x40 <- INA260 String B
=== M1 STEP 2+3 TEST ===
INA260 A (I2C1): OK
INA260 B (I2C3): OK
ALL OFF             | SW A:0 B:0 L:0 | A: 11.95V -2mA | B: 11.86V -3mA | rec 0/0 fail 0/0
STR_A on, load off  | SW A:1 B:0 L:0 | A: 11.95V -2mA | B: 11.86V -1mA | rec 0/0 fail 0/0
STR_A on, load ON   | SW A:1 B:0 L:1 | A: 11.95V -2mA | B: 11.86V -1mA | rec 0/0 fail 0/0
ALL OFF (A released) | SW A:0 B:0 L:0 | A: 11.95V -3mA | B: 11.86V -3mA | rec 0/0 fail 0/0
STR_B on, load off  | SW A:0 B:1 L:0 | A: 11.95V -1mA | B: 11.86V -3mA | rec 0/0 fail 0/0
STR_B on, load ON   | SW A:0 B:1 L:1 | A: 11.95V -2mA | B: 11.86V -2mA | rec 0/0 fail 0/0
--- cycle complete ---
```

Appendix B: Data Sheets

The tables below list the datasheet values used in this report. Cell ratings are given in Table 3.3.

Table B.1: Infineon IRF4905 specifications [12]

Parameter	Value	Condition
Drain-source breakdown voltage	−55 V	V_{GS} = 0 V
Continuous drain current	−74 A	T_C = 25 °C, V_{GS} = −10 V
Pulsed drain current	−260 A	—
On-resistance (max.)	0.02 Ω	V_{GS} = −10 V, I_D = −38 A
Gate threshold voltage	−2.0 to −4.0 V	V_{DS} = V_{GS}, I_D = −250 µA
Gate-source voltage (max.)	±20 V	—
Total gate charge (max.)	180 nC	I_D = −38 A
Drain-source leakage (max.)	−25 µA	V_{DS} = −55 V, V_{GS} = 0 V
Body-diode forward voltage (max.)	−1.6 V	I_S = −38 A
Junction-to-ambient thermal resistance	62 °C/W	—

Table B.2: onsemi P2N2222A specifications [13]

Parameter	Value	Condition
Collector-emitter voltage (max.)	40 V	—
Continuous collector current (max.)	600 mA	—
DC current gain (min.)	50	I_C = 1 mA, V_{CE} = 10 V
DC current gain	100 to 300	I_C = 150 mA, V_{CE} = 10 V
Collector-emitter saturation voltage (max.)	0.3 V	I_C = 150 mA, I_B = 15 mA

Table B.3: Texas Instruments INA260 specifications [14]

Parameter	Value
Integrated shunt resistance	2 mΩ
Continuous current	15 A
Bus-voltage range	0 to 36 V
Current register resolution	1.25 mA
Bus-voltage register resolution	1.25 mV
Supply voltage	2.7 to 5.5 V
Address with A1 = A0 = GND	0x40

Table B.4: STM32F446RE and Nucleo-64 board specifications [15, 16]

Parameter	**Value**
Core	Arm Cortex-M4 with FPU
Maximum clock frequency	180 MHz
Flash / SRAM (STM32F446RE)	512 KB / 128 KB
Nucleo VIN input range	7 to 12 V
Maximum VIN current for 9 V < VIN ≤ 12 V	250 mA
Debug and virtual COM port	On-board ST-Link

Appendix C: List of Components

Table C.1: List of components (from the PCB netlist)

Reference	Component	Function
U1	STM32 Nucleo-64 (STM32F446RE)	Controller
U2, U3	INA260 breakout modules	String A and string B sensing
U4	L298N dual H-bridge module	Motor driver
DS1	1.3-inch 128×64 I²C OLED	Local status display
Q1, Q2 / Q5, Q6 / Q8, Q9	IRF4905 P-channel MOSFET (×6)	String A, string B and load switch pairs
Q3, Q4, Q7	NPN transistor, 2N2222A family (×3)	Level shifters
D1, D2, D3	1N5400 diode (×3)	Series diode in each base path
R1, R4, R7	1 kΩ (×3)	Base resistors
R2, R5, R8	100 kΩ (×3)	Base pull-downs (default off)
R3, R6, R9	10 kΩ (×3)	Gate-source pull-ups
J1, J3	Two-way screw terminals	String A and string B connections
—	Samsung INR18650-30Q cells (×6)	Two 3S1P strings
—	Tracked chassis, two DC motors, kill switch	Rover platform

Appendix D: List of Papers Presented and Published

The following paper has been prepared from this project:

- H. Patil, N. Lad and A. Rodrigues, “A Fail-Safe String-Level Reconfigurable Battery Management System Using Discrete Back-to-Back P-MOSFET Switches: Design and Rover Validation on a 3S2P Li-Ion Pack.” Prepared in IEEE conference format, to be submitted.